TECHNICAL REVISION NOTICE

Prodify - Current Implementation Addendum

Purpose

This addendum is placed before the original project report and supersedes any earlier technical statements that conflict with the current repository implementation. The original report layout, figures, acknowledgements, and academic front matter are preserved unchanged after this notice.

Repository baseline

Verified against Prodify commit 39d37e4. Working tree was clean during review. The current implementation is the authoritative source for feature behavior, routes, dependency versions, and database semantics.

Most important correction

The camera/webcam and MediaPipe vision feature has been permanently removed. Prodify now uses local keyboard/mouse input and inactivity monitoring. Window intent classification is also fully local and does not call Groq.

How to read this report

Where the original report describes camera presence, /vision, camera_enabled, MediaPipe, or Groq as part of window classification, use the corrected descriptions in this addendum instead.

Local intent and input activity pipeline

Window intent classification

The active application and window title are evaluated by backend/app/services/intent_classifier.py. The pipeline applies transient operating-system filtering, session overrides, an LRU semantic cache, local Scikit-Learn TF-IDF similarity, cosine similarity, token overlap, and domain concept clusters. A strike-latch state machine converts verdicts into fair focus interruptions.

Offline guarantee for classification

classify_with_intent uses only in-process Python and Scikit-Learn for active-window verdicts. There is no Groq, OpenAI, network request, temperature setting, or remote fallback in this path. Generation counters discard stale classifications and an RLock protects shared classifier state.

Input activity monitoring

The frontend hook useIdleDetection records local keydown, pointer, and wheel activity. InputActivityMonitor.tsx displays live counts, idle duration, threshold state, and contextual alignment. The backend endpoint /api/telemetry/input-activity evaluates and persists lightweight telemetry when the API is reachable.

Contextual allowance

Aligned passive contexts such as video lectures, documentation, readers, tutorials, and research receive an extended inactivity allowance. Sustained inactivity outside that allowance pauses the focus timer locally and records an input-inactivity event.

Privacy boundary

No webcam stream, frame capture, face detector, biometric signal, camera permission, or /vision WebSocket exists in the current runtime.

Accurate offline and network wording

Database selection

backend/app/models/connection.py tests the configured PostgreSQL connection during startup. If that initial test fails, SQLAlchemy is configured against the local data/prodify.db SQLite file. This is a startup fallback, not a transparent runtime engine swap after a later PostgreSQL outage.

What works locally

The local intent classifier, timer behavior, input inactivity policy, SQLAlchemy SQLite persistence, and metric calculations can operate without an internet connection, subject to authentication/session availability and local database health.

What remains cloud-dependent

Supabase authentication, PostgreSQL synchronization, social features, and AI coaching can require network access. Groq remains an optional dependency for coaching/assistant paths, not for window classification.

Recommended report wording

Do not describe Prodify as entirely offline. Describe it as offline-capable for focus enforcement and local persistence, with cloud-dependent authentication, social synchronization, and AI coaching features.

Schema, deletion, and analytics facts

Camera schema

camera_enabled is no longer a Workspace column or workspace payload field. The Alembic migration d7e8f9a0b1c2_remove_camera_enabled.py removes the historical PostgreSQL/SQLite column.

Workspace deletion

DELETE /workspaces/{id} performs child-record cleanup and recomputes global streak state transactionally. By default, WorkspaceMetrics rows are deleted. With keep_metrics=true, metrics are retained and their workspace_id is set NULL through the model's ON DELETE SET NULL relationship.

Global streak

current_global_streak is explicitly reset to zero when no active workspace remains or an active track fails the current day's target. It is not correct to state that all metrics always disappear after deletion.

Timezone handling

metrics_engine.py uses ZoneInfo, converts local dates to UTC-naive query bounds, and buckets session timestamps in the session/user timezone. This is the correct basis for midnight-safe daily metrics; the report should avoid describing the system as server-time based.

Peak Hour

The current metrics peak-focus calculation is a trailing 14-day heuristic requiring 14 distinct local dates. It buckets session start hours and selects the best alignment rate. It should not be labeled as K-Means or generic clustering.

Use these versions in the final Black Book

Frontend

React 19.2.8; React DOM 19.2.8; TypeScript 7.0.2; Vite 8.2.0; Zustand 5.0.15; Tailwind CSS 3.4.17; Tauri 2.11.x.

Backend

FastAPI 0.111.0; SQLAlchemy 2.0.30; Alembic >=1.13,<2.0; Scikit-Learn >=1.4.2,<2.0; psycopg2-binary 2.9.10; Python dotenv 1.2.2; Uvicorn 0.29.0.

AI boundary

Groq/OpenAI-compatible client code remains in AI coaching and assistant modules only. It must not be listed as a dependency of the telemetry intent-classification algorithm.

Removed dependencies

MediaPipe, OpenCV, react-webcam, camera permission handling, and the vision model asset are removed from the production implementation.

Build verification

The frontend production build passes with only existing non-fatal Vite dynamic-import warnings. Backend Python compilation and local classifier/input-policy smoke tests pass.

A Project Report

on

PRODIFY (AI-POWERED ADAPTIVE FOCUS AND PRODUCTIVITY SYSTEM)

Submitted By

PRANAV PALANISWAMY

(Roll No. 202403056)

in Partial Fulfillment for the Award of the Degree of

BACHELOR OF SCIENCE

in

COMPUTER SCIENCE

Under the guidance of

Mr. AJIT SHARMA

DEPARTMENT OF COMPUTER SCIENCE



ZSCT'S THAKUR SHYAMNARAYAN DEGREE COLLEGE

Sem V

2026-2027



ZSCT'S THAKUR SHYAMNARAYAN DEGREE COLLEGE, Kandivali East
MUMBAI - 400101
2026-2027

DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

This is to certify that Mr. **Pranav Palaniswamy** of **T.Y.B.Sc (Sem V)** class has satisfactorily completed the project "**PRODIFY**", to be submitted in partial fulfilment for the award of degree of **Bachelor of Science** in **Computer Science** during the Academic year **2026-2027**.

Date of Submission:

Project Guide

Head/Incharge,
Department Of Computer Science

College Seal

Signature of Examiner

DECLARATION

I, Pranav Palaniswamy, hereby declare that the project entitled, "**PRODIFY (AI-Powered Adaptive Focus and Productivity System)**" submitted in the partial fulfillment for the award of **Bachelor of Science** in **Computer Science** during the academic year 2026 – 2027 is my original work and the project has not formed the basis for the award of any degree, associateship, fellowship or any other similar titles.

Signature of the Student:

Place:

Date:

ACKNOWLEDGEMENT

I express my sincere gratitude to all those who have contributed to the successful completion of the **Prodify (AI-Powered Adaptive Focus and Productivity System)** project.

First and foremost, I would like to express my sincere gratitude to our esteemed principal, **"Dr. G.D. Giri"**, whose unwavering support and inspirational leadership served as the foundation of this project. His vision and dedication have been a constant source of motivation. I am highly indebted to our Co-Coordinator, **"Mrs. Abha Dhote"**, and to my mentor and project guide, **"Mr. Ajit Sharma"**, for their invaluable support and guidance throughout my project. Their encouragement and insightful feedback have been instrumental in shaping my work and helping me develop my skills. Thank you for believing in me and for your unwavering dedication to your students.

I am grateful to my friends and family for their constant encouragement. Their understanding during the long and demanding phases of development, testing and debugging was a continuous source of motivation.

Special appreciation goes to the open-source community and the maintainers of the tools on which Prodify is built, including Tauri, React, FastAPI, SQLAlchemy, Scikit-Learn, SQLite and PostgreSQL. The documentation, forums and collaborative platforms they maintain made it possible to overcome many technical challenges and to find practical solutions.

Last but not least, I thank everyone who tried early versions of Prodify and shared feedback. Their observations on false alerts, unclear metrics and interface problems directly improved the final system.

This project has been a rewarding journey, and I am thankful for the collective effort that has gone into making Prodify a reality.

ABSTRACT

Prodify is an AI-powered adaptive focus and productivity system delivered as a desktop application for Windows. It helps students, developers and remote workers sustain deep work by comparing what they are actually doing on their computer with the goal they declared for the work session. Unlike conventional Pomodoro timers and website blockers, which rely on fixed lists of allowed and blocked applications, Prodify judges whether the currently active window is relevant to the user's stated intent. No executable is ever approved or rejected by its name alone: the same web browser that is a distraction during one session can be a legitimate research tool in the next.

The user organises work into workspaces. Each workspace belongs to one of three tracks: **Mastery** (long-term daily habit commitments), **Sprint** (deadline-driven work whose daily quota is calculated from the target hours and the deadline) and **Quick Work** (ad-hoc tasks with a daily target of zero, which log volume and purity without ever triggering unfulfilled-quota penalties or breaking a streak). During a focus session the application observes the active window and sends the application name and window title, together with the declared goal, to a hybrid semantic intent engine. The engine first filters transient operating-system windows, then consults a session verdict cache and the user's instant "Allow for Session" whitelist, then computes an offline multi-scale vector similarity with Scikit-Learn (TF-IDF over word (1,3) and character (3,5) n-grams, cosine similarity and domain concept clusters) and finally, for borderline cases, asks a Groq-hosted large language model (qwen3 / Llama family, temperature zero) for a verdict with a short human-readable reason. A strike-based state machine (aligned, pending, distracted) turns these verdicts into fair interruptions.

Prodify replaces vanity statistics with five honest metrics computed on real data in the user's local time zone: **Today's Progress**, **7D Consistency**, **Focus Purity**, **Peak Hour (14D Model)** and **Active Streak**. A Report page provides a Workspace Performance Leaderboard, an interactive Distraction Intelligence panel and a structured goal calendar with a day inspector. The **Star ML** predictive engine trains Scikit-Learn Random Forest models on the user's own session history to estimate a personal Attention Decay Wall, the chronobiological peak hour and a Burnout Risk Score, and feeds these findings into the AI coach.

A privacy-first social module adds a "My Circle & Chat" tab with live presence, *@username* handles, friend requests, anonymised track summaries and a real-time direct-message drawer, and a "Global Leaderboard" tab with worldwide rankings and Top-3 podium cards. Workspace deletion follows a hard-cascade, zero-orphan architecture, so that no sessions, logs or metrics survive a deleted workspace.

Technologically, the front end is built with Tauri v2 (Rust), React 18, TypeScript, Vite and Tailwind CSS, with Zustand for persistent timer state and a reactive cache store for optimistic updates. The back end is a FastAPI service using SQLAlchemy 2.0 and Alembic on a Supabase-hosted PostgreSQL database with an automatic SQLite local fallback, with Supabase Authentication and JSON Web Tokens for per-user data isolation. Optional presence detection uses Google MediaPipe. The interface follows a unified dark "Vercel Monolith" design system.

In conclusion, Prodify demonstrates how an offline semantic classifier, a large language model, a transparent state machine, personal machine-learning models and carefully defined metrics can be combined to build a focus tool that understands context, treats the user fairly and reports progress honestly.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION 1

1.1 Introduction 2

1.2 Objectives 2

1.3 Purpose, Scope, Architecture & Components 4

1.3.1 Purpose 4

1.3.2 Scope 4

1.3.3 Architecture & Components 5

1.3.4 Summary 8

CHAPTER 2 SURVEY OF TECHNOLOGY 9

2.1 Problem Definition 10

2.2 System Requirements 11

2.2.1 Hardware Requirements 11

2.2.2 Software Requirements 11

2.3 Justification of Selection of Technology 11

2.4 Planning and Scheduling 13

2.4.1 Gantt Chart 13

CHAPTER 3 SYSTEM DESIGN 15

3.1 UML Diagrams 16

3.1.1 Sequence Diagram 16

3.1.2 Use Case Diagram 17

3.1.3 State Diagram 18

3.1.4 Activity Diagram 19

3.1.5 Data Flow Diagram (DFD) 20

3.1.6 Deployment Diagram 21

3.2 Metrics Engine Design 21

Workspace Tracks and Daily Targets 21

Day Status 21

Timezone-Aware Aggregation 22

The Five Home Command Strip Metrics 22

Consistency Score Components 23

Report Page (Report.tsx) . . . 23
3.3 Intent Engine Design . . . 23
3.4 Star ML Predictive Engine Design . . . 24
3.5 Social Circle and Hard Cascade Deletion Design . . . 24

CHAPTER 4 IMPLEMENTATION & TESTING . . . 25
4.1 Code . . . 26
4.2 Test Case . . . 28
Test Case 1: Authentication and profile loading . . . 28
Test Case 2: Workspace creation (three tracks) . . . 28
Test Case 3: Focus timer and session counter . . . 29
Test Case 4: Distraction detection with the hybrid engine . . . 30
Test Case 5: Metrics computation . . . 31
Test Case 6: AI coach conversation with Star ML insights . . . 31
Test Case 7: Hard cascade workspace deletion and zero-orphan verification . . . 32
Test Case 8: Social circle, handle search and real-time peer chat . . . 33
4.3 Testing Approach . . . 33

CHAPTER 5 RESULT . . . 36
Screenshots of Prodify . . . 37
Interruptions and Calendar . . . 38
AI Coach and Settings . . . 38
Reports . . . 39
Social Circle and Global Leaderboard . . . 39

CHAPTER 6 CONCLUSION . . . 41
6.1 Conclusion . . . 42
6.2 Future Scope . . . 43

CHAPTER 7 LIST OF CONTENTS . . . 44

CHAPTER 8 REFERENCES . . . 46
8.1 References for Prodify . . . 47
1. Books and Research Papers . . . 47
2. Software and API Documentation . . . 47

3. Standards 48

CHAPTER 9 GLOSSARY AND APPENDICES 49

9.1 Glossary of Prodify 50

9.2 Appendix A: Data Model / ER Diagram 52

100% Hard Cascade Deletion & Zero-Orphan Architecture 52

Table: users 53

Table: workspaces 53

Table: sessions 54

Table: activity_logs 54

Table: telemetry_logs 55

Table: daily_workspace_logs 55

Table: user_stats 55

Table: workspace_metrics 55

Table: friendships 56

Table: direct_messages 56

9.3 Appendix B: Operation and Protocol 56

B.1 Main API Endpoints 57

B.2 Sign-in Protocol 57

B.3 Activity and Session Protocol 58

B.4 Use Case Scenarios 58

B.5 Intent Classification Process 58

B.6 Security and Privacy Measures 58

B.7 Third-Party Services and Libraries 59

PLAGIARISM REPORT

[Paste the plagiarism / similarity report screenshot here]

CHAPTER 1
INTRODUCTION

1.1 Introduction

Prodify is an AI-powered adaptive focus and productivity system built as a desktop application. It is designed for people whose work happens on a computer – students preparing assignments, developers building projects, designers, writers, researchers and remote professionals – and who find that their attention is constantly pulled away by notifications, social media, video platforms and the many tabs and windows that a modern computer offers. Prodify helps them start a focused work session, stay inside the goal they have set for that session, and understand honestly how well they are doing over days and weeks.

Most existing tools approach this problem with static rules. A Pomodoro timer only counts minutes, and a website blocker only compares a process name or web address with a fixed list. Both ignore context: a video site may be the best study resource for an art student following a figure-drawing tutorial and a pure distraction for someone writing a report, and a browser may be a developer's documentation reader or a source of memes. Prodify solves this by judging each active window against the goal the user typed in plain English, following an *Executable Neutrality Principle*: browsers, media players and editors are universal canvases and are judged by what they show, never by their binary name.

Around this semantic core, Prodify provides the features needed for a complete focus routine. Work is organised into workspaces, each with its own goal, daily target, break lengths and colour theme, and each running as a **Mastery**, **Sprint** or **Quick Work** track. A timer manages focus sessions and breaks and remembers its state across restarts. When the user leaves the goal, a fair two-strike mechanism warns first and interrupts second, always showing the reason. Optional camera-based presence detection and idle detection add further evidence. An AI coach, informed by a personal Star ML model, answers questions and comments on each finished session, and a privacy-first social circle adds friendly accountability.

Prodify also rethinks how progress is measured. Instead of showing total hours, which reward sitting at the desk rather than working, it computes five metrics – Today's Progress, 7D Consistency, Focus Purity, Peak Hour (14D Model) and Active Streak – from real data in the user's own time zone, and a Report page shows where time is lost and when the user concentrates best.

Technically, Prodify is a full-stack system. The desktop client uses Tauri v2 with a Rust core and a React and TypeScript interface; the back end is a FastAPI service backed by PostgreSQL on Supabase with a SQLite local fallback. Authentication uses Supabase Auth with JSON Web Tokens, so that every user can read and change only their own data. Semantic classification combines an offline Scikit-Learn vector engine with a Groq-hosted language model, and behavioural prediction uses Scikit-Learn Random Forest models.

1.2 Objectives

The primary objective of Prodify is to build a context-aware focus assistant that helps users stay aligned with their own goals, treats them fairly, and reports their progress honestly. The key objectives of the system are:

1. Detect Distraction by Meaning, Not by Lists

- Judge every active window against the goal the user declared, using a hybrid of offline vector similarity and a large language model over the application name and window title.

- Remove the need to maintain allow-lists and block-lists, so that the same tool works for studying, coding, drawing, writing or research.

2. Interrupt Fairly and Transparently

- Use a strike-based state machine so that a single accidental window switch never ends a session.
- Always display a short reason for an interruption and let the user respond with “Back to Focus” or “Allow for Session”.

3. Organise Work into Three Goal-Oriented Workspace Tracks

- **Mastery** tracks for long-term daily habits with a fixed daily quota; **Sprint** tracks for deadline-driven work with a dynamic daily quota computed from the target hours and the days remaining; and **Quick Work** tracks with `daily_target = 0` for ad-hoc, streak-safe tasks.
- Give each workspace its own goal, timer durations and colour theme.

4. Provide Meaningful and Honest Metrics

- Compute Today’s Progress, 7D Consistency, Focus Purity, Peak Hour (14D Model) and Active Streak from real logged data only.
- Define every day in the user’s local time zone, and show “Calibrating <14d” or “not enough data yet” instead of invented numbers.

5. Predict Personal Attention and Burnout Patterns

- Train Random Forest models on the user’s own sessions to find the Attention Decay Wall, the chronobiological peak hour and a Burnout Risk Score.
- Feed these findings to the AI coach so that its advice is specific, and give short feedback after each session.

6. Promote Healthy Work Habits

- Provide breaks, long breaks and break-over notifications so that the user rests as deliberately as they work, and warn about overwork.

7. Add Privacy-First Social Accountability

- Provide handles, friend requests, live presence, anonymised track summaries, real-time direct messages and a global leaderboard without ever exposing window titles or workspace names.

8. Guarantee Data Integrity

- Delete a workspace with a hard cascade so that zero orphaned rows or ghost metrics survive, and recompute dependent metrics immediately.

9. Protect Security and Privacy

- Authenticate users with Supabase Auth and JSON Web Tokens and isolate each user’s data; record only application names and window titles; keep camera tracking optional and off by default; and let users erase all their data.

10. Deliver a Lightweight, Responsive and Consistent Desktop Experience

- Use a native Tauri shell for a small installer and access to operating-system features, a reactive cache store for instant updates, and one unified dark design system across all pages.

By achieving these objectives, Prodify aims to become a trustworthy companion for focused work: one that understands context, explains itself, and measures progress in a way that users can believe.

1.3 Purpose, Scope, Architecture & Components

1.3.1 Purpose

The purpose of Prodify is to help people begin and sustain focused work, and to give them an honest picture of their consistency. It targets the gap between a timer that only counts minutes and a blocker that only compares names: the user wants to be reminded when they drift away from their goal, but does not want to be punished for using the tools their goal requires.

For users, Prodify acts as a calm accountability partner. They declare what they are working on, press start, and are interrupted only when the work on screen no longer matches that statement. Afterwards they see how the day compared with their target, at which hours they concentrate best, how long they can focus before attention decays, and what the AI coach suggests for the next session.

From a technical perspective, Prodify demonstrates how a modern desktop application can combine a native shell, a web-based interface, a cloud API, a relational database, classical machine learning and a hosted large language model into one coherent product. It uses the language model as one stage of a deterministic pipeline: cheap rules, a cache and an offline vector engine handle most decisions, while a state machine keeps behaviour predictable and fast.

Privacy is a fundamental consideration. Prodify stores the name of the active application and its window title, but not screen contents. Camera-based presence detection is optional, is switched on per workspace and is off by default. Social views are anonymised, and the Settings page allows all personal data to be deleted at any time.

1.3.2 Scope

The scope of the present version of Prodify covers the following areas, all of which are implemented.

- **Platform:** a Windows desktop application built with Tauri v2, with a hosted back-end API and a web client.
- **Accounts:** email and Google sign-in through Supabase Auth, with user profile, unique *@username* handle and consent management.
- **Workspaces:** creation, editing and deletion of Mastery, Sprint and Quick Work workspaces with goals, daily commitments, timer settings, allowed-task descriptions and colour themes.
- **Focus sessions:** focus timer, short and long breaks, automatic and manual pausing, idle detection, break-over notifications and sound alerts.
- **Hybrid intent engine:** transient-window filtering, session cache with “Allow for Session” override, offline TF-IDF vector similarity, Groq LLM verdicts and a strike-based state machine.
- **Metrics and reports:** the five Home metrics, streaks, the structured goal calendar with day inspector, and the Report page with the Workspace Performance Leaderboard and Distraction Intelligence.
- **Star ML predictive engine:** a personally trained machine-learning model (Random Forest) for Attention Decay Wall, Peak Hour and Burnout Risk, feeding the AI coach.
- **Social features:** friends, handle search, requests, live presence, anonymised track summaries, real-time direct chat and a global leaderboard.
- **Offline-resilient operation:** the offline semantic classifier keeps window judgement available without the cloud model, and an automatic SQLite local fallback keeps data operations working

when the cloud database is unreachable.

- **Data integrity and wellbeing:** hard cascade workspace deletion, burnout-risk estimation and distraction reports.

The following are outside the scope of this version and are described as future scope in Chapter 6: mobile (iOS and Android), macOS and Linux clients; a browser DOM extension; operating-system level kernel focus mode; wearable biometric integration; and calendar integration. Features that depend on the cloud (the language-model verdict, social chat and the global leaderboard) naturally require an internet connection.

1.3.3 Architecture & Components

Prodify follows a layered client–server architecture made up of four parts: the desktop client, the application server, the data layer and the external services. Each layer has a clearly defined responsibility, which keeps the system maintainable and allows each part to evolve independently.

1. Architecture Overview

- **Presentation Layer:** Tauri desktop client (React 18 and TypeScript interface, Rust native core)
- **Application Layer:** FastAPI back end (REST API, intent engine, metrics engine, Star ML engine, social module, vision service)
- **Data Layer:** Supabase PostgreSQL with SQLite local fallback, managed with SQLAlchemy 2.0 and Alembic
- **External Services and Libraries:** Supabase Auth, Groq API (qwen3 / Llama models), Scikit-Learn and Google MediaPipe

1.1 Presentation Layer (Desktop Client)

The client is the part the user sees and also the sensor that observes the active window. Technologies: Tauri v2, Rust, React 18, TypeScript, Vite, Tailwind CSS, Zustand.

- **User Interface:** pages for sign-in, Home dashboard (Focus Command Strip), workspace timer, Report, Friends and Settings, with modal dialogs for workspace creation, timer settings and distraction reports. A unified “Vercel Monolith” dark design system is used throughout.
- **Native Core (Rust):** reads the active application and window title, sends operating-system notifications, handles the deep link that completes Google sign-in and brings the window to the front when a break ends.
- **Timer Store:** a persisted Zustand store that holds the state of each workspace timer (idle, focus running, paused, break running, completed), session counters and distraction counters.
- **Cache Store (`cacheStore.ts`):** a reactive TTL cache with optimistic updates, background polling synchronisation and instant invalidation after create, update and delete operations.
- **Adaptive Focus Hook:** polls the active window, sends it to the back end and reacts to the returned focus state by pausing the timer and showing the interruption dialog.
- **Metric Popovers (`MetricInfoPopover.tsx`):** single-open educational popovers that explain every Home metric.
- **AI Coach Panel:** a chat interface that sends messages together with live session context and shows the coach’s replies.
- **Friends Page (`FriendsPage.tsx`):** the social module described in Section 1.3.3 (3).

1.2 Application Layer (Back End)

The back end holds all business logic and is the only component that talks to the database and to the language model. Technologies: Python 3.12, FastAPI, SQLAlchemy 2.0, Pydantic, Scikit-Learn, NumPy.

- **Authentication and Authorisation:** validates Supabase JWTs and resolves the current user for every request, so that each query is restricted to that user's data.
- **Telemetry API:** receives activity events and completed sessions.
- **Intent Engine (`intent_classifier.py`):** a four-stage hybrid classifier, described below.
- **Metrics Engine (`metrics_engine.py`):** computes the five Home metrics, streaks, report data and calendar statuses on demand, with timezone-aware aggregation.
- **Star ML Engine (`star_ml.py`):** trains Scikit-Learn `RandomForestClassifier` and `RandomForestRegressor` models on the user's historical sessions to compute the personal Attention Decay Wall (the minute mark where focus degrades), the Chronobiological Peak Hour (14D model) and the Burnout Risk Score, and injects these findings into the AI coach prompt.
- **Social Module (`friends.py` and leaderboard routes):** handle search, friend requests, presence, anonymised track summaries, direct messages and the global ranking.
- **Vision Service:** optional WebSocket endpoint that analyses camera frames for face presence using MediaPipe.

Hybrid Semantic Intent Engine. The classifier in `intent_classifier.py` runs the following stages and stops at the first one that gives a decision: (1) **Transient OS window filtering** ignores shell windows such as the task switcher and empty titles; (2) **Session Verdict Cache and "Allow for Session" override** returns an earlier verdict for the same goal, application and title, or accepts a window the user has whitelisted for the session; (3) **Offline Scikit-Learn Multi-Scale Vector Similarity** computes TF-IDF cosine similarity between the declared goal and the window title using `TfidfVectorizer` with word (1,3) n-grams and character (3,5) n-grams, expanded by domain concept clusters, and works without any network access; and (4) the **Groq API language model** (qwen3 / Llama family) is asked, at temperature zero, for a contextual verdict with a one-line summary and reason when the offline score is borderline or the cache misses.

1.3 Data Layer (Database)

The data layer stores all persistent information. Technologies: PostgreSQL on Supabase with an automatic SQLite local fallback, SQLAlchemy ORM, Alembic migrations. Foreign-key enforcement is switched on for SQLite (`PRAGMA foreign_keys = ON`) so that cascade rules behave identically in both back ends.

- **User and Workspace Data:** profiles, handles, workspaces and their configuration, and `user_stats`.
- **Session Data:** sessions with start and end time, duration, focused and distracted seconds, purity score, burnout score and time zone.
- **Activity and Telemetry Logs:** application name, window title, verdict and timestamp for each classified window.
- **Metrics and Coaching Data:** daily workspace logs, workspace metrics and coach chat history.
- **Social Data:** friendships and direct messages.

2. Friends & Global Leaderboard Module

The social module (`FriendsPage.tsx` on the client, `friends.py` on the server) is split into two tabs:

- **Tab 1 "My Circle & Chat":** a pinned status card shows the user's live state ("Focusing now", "Online" or "Offline"), the *@username* handle editor, active streak and 7D Consistency. The focus-circle table lists connected friends with live status, streak, momentum and **anonymised track summaries** such as "2 Mastery · 1 Quick Work", never exposing private window titles or workspace names. Peer discovery searches by handle, name or email, and requests can be accepted or declined in one click. A slide-over **Direct Messaging Chat Drawer** provides real-time peer-to-peer messages with a three-second polling loop and auto-scroll.
- **Tab 2 "Global Leaderboard":** worldwide ranking ordered by Active Streak and then 7D Consistency (`momentum_score`), with Top-3 podium cards (gold, silver, bronze), a searchable table and direct "Add Friend" and "Chat" buttons.

3. System Components

- **3.1 User Authentication:** email and Google sign-in with token-based sessions; a consent screen on first use.
- **3.2 Workspace Management:** create, edit and delete Mastery, Sprint and Quick Work workspaces; choose a daily commitment, durations, a colour and a description of allowed apps and tasks.
- **3.3 Focus Timer:** focus, short break and long break modes; automatic or manual start of breaks and sessions; persistent state.
- **3.4 Distraction Handling:** interruption dialog with "Allow for Session", sound and notification alerts, distraction counts and the interactive Distraction Intelligence report.
- **3.5 Metrics and Reports:** Focus Command Strip, Workspace Performance Leaderboard, structured goal calendar and day inspector.
- **3.6 AI Coaching and Star ML:** chat and post-session debrief with up to 60 days of history and personal model findings.
- **3.7 Friends and Global Leaderboard:** presence, requests, anonymised summaries, direct chat and ranking.
- **3.8 Data Integrity:** hard cascade workspace deletion with immediate metric recomputation and cache invalidation.
- **3.9 Security and Privacy:** JWT validation, user-level data isolation, consent, optional camera and full data reset.
- **3.10 Settings:** global timer settings (auto-start, long-break interval, alarm and focus sound, hour format) that apply to every workspace.

4. Scalability and Performance

- **Stateless API:** the FastAPI service keeps no session state, so it can be replicated when more users arrive.
- **Verdict Caching:** classification results are cached by a hash of goal, application and title, so a repeated window costs no computation or model call.

- **Offline First:** the offline vector stage resolves most windows locally, so the language model is called only for borderline cases.
- **Throttled Polling and Client Cache:** the client sends window changes rather than every poll, and the TTL cache avoids repeated requests.

The architecture of Prodify separates sensing, judgement, prediction, storage and presentation. This makes it possible to improve the classifier, change the metrics or redesign the interface without rewriting the rest of the system.

1.3.4 Summary

Prodify is a desktop focus system that understands what the user is trying to do. It combines a Tauri and React client, a FastAPI back end, a PostgreSQL database with a SQLite fallback, an offline Scikit-Learn engine and a hosted language model to detect distraction by meaning, to interrupt fairly and to report progress honestly. Its three workspace tracks, timer, hybrid intent engine, metrics, Star ML coach and social circle work together to give users both real-time protection of their attention and long-term insight into their habits.

The system was built with security, privacy and honesty in mind: users are authenticated and isolated, tracking is limited to what is needed, optional features are off by default, social data is anonymised and metrics are computed only from real data. The remaining chapters describe the technologies chosen, the design of the system, its implementation and testing, the results obtained, and the directions in which it can grow.

CHAPTER 2
SURVEY OF TECHNOLOGY

2.1 Problem Definition

Modern knowledge work happens on screens that are designed to compete for attention. Students and professionals who sit down to work are a click away from social media, video platforms and messaging, and every interruption has a cost. Research on interrupted work shows that people compensate for interruptions by working faster, but at the price of higher stress and frustration, and that fragmented work days are the norm rather than the exception. The tools that exist to protect focus have important weaknesses. The key problems identified in the current focus and productivity landscape are:

1. **Context-Blind Blocking:** website and application blockers use hardcoded process blacklists. They cannot tell whether a browser tab or video is a lecture or entertainment, so a developer reading API documentation, an art student watching a figure-drawing tutorial on YouTube or a researcher watching an archival documentary in VLC is wrongly struck, while memes in an "approved" application go unnoticed.
2. **Heavy Configuration:** allow-lists and block-lists must be written and maintained by the user for every kind of work, and they quickly go out of date.
3. **Timers Without Awareness:** Pomodoro-style timers count minutes but cannot tell whether those minutes were spent on the task.
4. **Unfair or Opaque Interruptions:** tools that react to a single window switch, or that block without explanation, frustrate users and are soon disabled, which makes their analytics inaccurate.
5. **Vanity Metrics:** totals such as "hours logged" reward being present, not working well, and do not show whether the user is on track for their own goals.
6. **One-Size-Fits-All Targets:** long-term habits, deadline projects and quick ad-hoc tasks are treated identically, so casual tasks break streaks or inflate penalties.
7. **Invisible Overwork and Attention Limits:** long uninterrupted sessions late at night lead to fatigue and burnout, and most tools do not tell users how long they can really concentrate or when they are at their best.
8. **Poor Feedback:** users are rarely told which applications distract them most, at what time of day, or what to change.
9. **Privacy Concerns:** monitoring tools may record far more than needed, social tools may expose private activity, and users are rarely given control over their data.
10. **Generic Advice and No Accountability:** productivity apps give the same tips to everyone instead of adapting to an individual's goals and history, and offer no private way to share progress with friends.

Prodify addresses these problems by judging each window against the user's own declared goal, by interrupting only after a confirmed deviation and always explaining why, by computing metrics against personal targets with track-aware quotas, by learning personal attention and burnout patterns, by anonymising social data, and by keeping data collection minimal and under the user's control.

2.2 System Requirements

System requirements are the configuration that a system must have in order for a hardware or software application to run smoothly and efficiently. Failure to meet these requirements can result in installation problems or in poor performance. The requirements below are given for the user's machine (running the Prodify desktop application) and for the development machine used to build it.

2.2.1 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 / AMD Ryzen 3 (64-bit)	Intel Core i5 / AMD Ryzen 5 or better
RAM	4 GB	8 GB or more
Storage	500 MB free space	2 GB free space (development: 10 GB)
Display	1366 × 768	1920 × 1080
Camera	Not required	Webcam (only for optional presence detection)
Internet	Required for the cloud API, language model, social features and cloud database; the offline classifier and SQLite fallback work without it	Broadband connection
System type	64-bit operating system, x64-based processor	64-bit operating system, x64-based processor

2.2.2 Software Requirements

Software	Purpose
Windows 10 / 11 (64-bit)	Operating system for the desktop client; requires the Microsoft Edge WebView2 runtime used by Tauri
Node.js (LTS) and npm	Builds and runs the React, TypeScript and Vite front end
Rust toolchain and Tauri CLI	Builds the native Tauri shell and the Windows installer
Python 3.12	Runs the FastAPI back end
Scikit-Learn (scikit-learn) and NumPy	Offline TF-IDF semantic vector similarity (TfidfVectorizer, cosine similarity) and Random Forest behavioural and burnout modelling (Star ML)
PostgreSQL (Supabase)	Managed relational database and authentication provider
SQLite	Automatic local fallback database for offline resilience and development
WSL 2 (Ubuntu)	Linux environment in which the back end is run during development
Visual Studio Code	Primary code editor
Git and GitHub	Version control and source hosting; pushes trigger deployment
Railway	Cloud platform that hosts the back-end service
Web browser	Used for Google sign-in, which returns to the app through a deep link

2.3 Justification of Selection of Technology

1. Tauri v2 and Rust for the Desktop Shell

- Small installers and low memory use compared with bundling a full browser engine, which suits an application that runs all day beside the user's real work.

- Rust gives safe, fast access to operating-system functions such as reading the active window, notifications and deep links.

2. React, TypeScript, Vite and Tailwind CSS for the Interface

- Component-based structure promotes reuse and maintainability across the dashboard, timer, report, friends page and dialogs.
- TypeScript catches data-shape errors at build time, which matters in an application with many API payloads.
- Tailwind CSS allows one consistent design system without large style sheets.

3. The "Vercel Monolith" Dark Design System

- A single dark-only engineering palette is used on every page: a deep obsidian canvas (`#09090b`), translucent zinc surfaces with `1px border-zinc-800` structural borders, `Inter` for structure and `JetBrains Mono` (`font-mono`) for all telemetry values, timers, ratios and handles.
- Matte white primary buttons give high contrast, and status colours carry meaning: emerald for aligned focus, amber for streaks and warnings, rose for distraction interventions. This keeps dense telemetry readable and the interface visually calm.

4. Zustand and a Reactive Cache Store for State

- A small, simple store with a persistence middleware allows timer state to survive restarts and page refreshes.
- The TTL cache store provides optimistic updates and instant invalidation after create and delete operations, so that deleted data disappears from the interface immediately.

5. FastAPI and Pydantic for the Back End

- Asynchronous, high-performance framework that suits many small, frequent telemetry requests and short polling loops for chat.
- Pydantic validates every request body, and interactive API documentation is generated automatically.

6. SQLAlchemy, Alembic, PostgreSQL and SQLite for Data

- Relational structure with ACID guarantees fits users, workspaces, sessions and friendships, supports foreign-key cascades and the aggregate queries used by the metrics engine.
- Alembic migrations make schema changes repeatable and reviewable.
- **SQLite local fallback**: the same SQLAlchemy models run on a local SQLite file when the cloud database is unreachable, which gives offline resilience and a zero-setup development database.

7. Scikit-Learn and NumPy for Offline Semantics and Behavioural Modelling

- `TfidfVectorizer` with word (1,3) and character (3,5) n-grams and cosine similarity gives a fast, explainable, fully offline measure of how close a window title is to the declared goal, and tolerates typos and mixed-language titles.
- `RandomForestClassifier` and `RandomForestRegressor` model non-linear relationships between session length, time of day and slips with small data sets and little tuning, which suits per-user training on a laptop.
- Running locally means that no personal session history is sent to a third party for modelling.

8. Supabase for Hosting and Authentication

- Managed PostgreSQL together with email and Google sign-in and signed JSON Web Tokens, so that password handling is left to a specialised service.

9. Groq and the qwen3 / Llama Models for Contextual Intelligence

- Very low inference latency makes it practical to classify borderline windows as soon as the user switches to them.
- Large language models understand abbreviations, informal goals and mixed-language window titles that fixed rules cannot.
- Running the classifier at zero temperature makes its verdicts repeatable, and the same models power the AI coach.

10. Google MediaPipe for Presence Detection

- Lightweight, well-tested face detection that can run on an ordinary laptop without a GPU.

11. Railway, Git and GitHub for Delivery

- Pushing to GitHub redeploys the back end automatically, which keeps releases quick and repeatable.

12. Visual Studio Code and WSL 2 as the Development Environment

- A lightweight editor with an integrated terminal, and a Linux environment that matches the production back end.

2.4 Planning and Scheduling

Every successful software project begins with a plan. Planning and scheduling are pivotal in project management: they start with a clear definition of scope, divide the work into tasks, sequence the tasks and estimate the time and resources they need. Prodify was planned as eight overlapping phases over twelve weeks, so that the data model and the intent engine – the parts on which everything else depends – were built and tested first, and the interface, metrics, machine-learning and social features were added on top of them. After each phase the work was reviewed, which informed the planning of the next phase.

2.4.1 Gantt Chart

A Gantt chart, commonly used in project management, is one of the most popular and useful ways of showing activities displayed against time. On the left of the chart is a list of the activities and along the top is a suitable time scale. Each activity is represented by a bar; the position and length of the bar reflects the start date, duration and end date of the activity. This allows us to see at a glance:

- What the various activities are
- When each activity begins and ends, and how long it is scheduled to last
- Where activities overlap with other activities, and by how much
- The start and end date of the whole project

Ph.	Activity	W 1	W 2	W 3	W 4	W 5	W 6	W 7	W 8	W 9	W 10	W 11	W 12
I	Requirement analysis & planning												
II	Architecture & database design												
III	Backend API & authentication												
IV	Hybrid intent engine & state machine												
V	Desktop client, timer & workspace tracks												
VI	Metrics engine, Report page & calendar												
VII	Star ML coach, social circle & leaderboard												
VIII	Testing, cascade verification, deployment & documentation												

Fig. 2.1 Prodify twelve-week development schedule

CHAPTER 3
SYSTEM DESIGN

3.1 UML Diagrams

The Unified Modeling Language (UML) is a standard way of visualising the design of a system. This section presents the diagrams that describe Prodify from the point of view of its behaviour over time, its users, its states and activities, the flow of its data and its deployment. The data model of the database is given as an entity-relationship diagram in Appendix A (Section 9.2).

3.1.1 Sequence Diagram

A Sequence Diagram visualises the interactions between objects or components of a system over time. It focuses on the chronological order of messages and shows how the parts collaborate to carry out one scenario. The main scenario in Prodify is the classification of a window during a focus session. The client reads the active window and sends it to the back end, which validates the token, filters transient windows and looks up the session cache and whitelist. On a miss, the offline vector engine computes a similarity score; only for a borderline score is the language model asked. The verdict is stored, the telemetry is logged, the focus state is updated and the state is returned with its reason. When the state is distracted, the client pauses the timer and shows the interruption dialog (messages 1 to 15 in Figure 3.1).

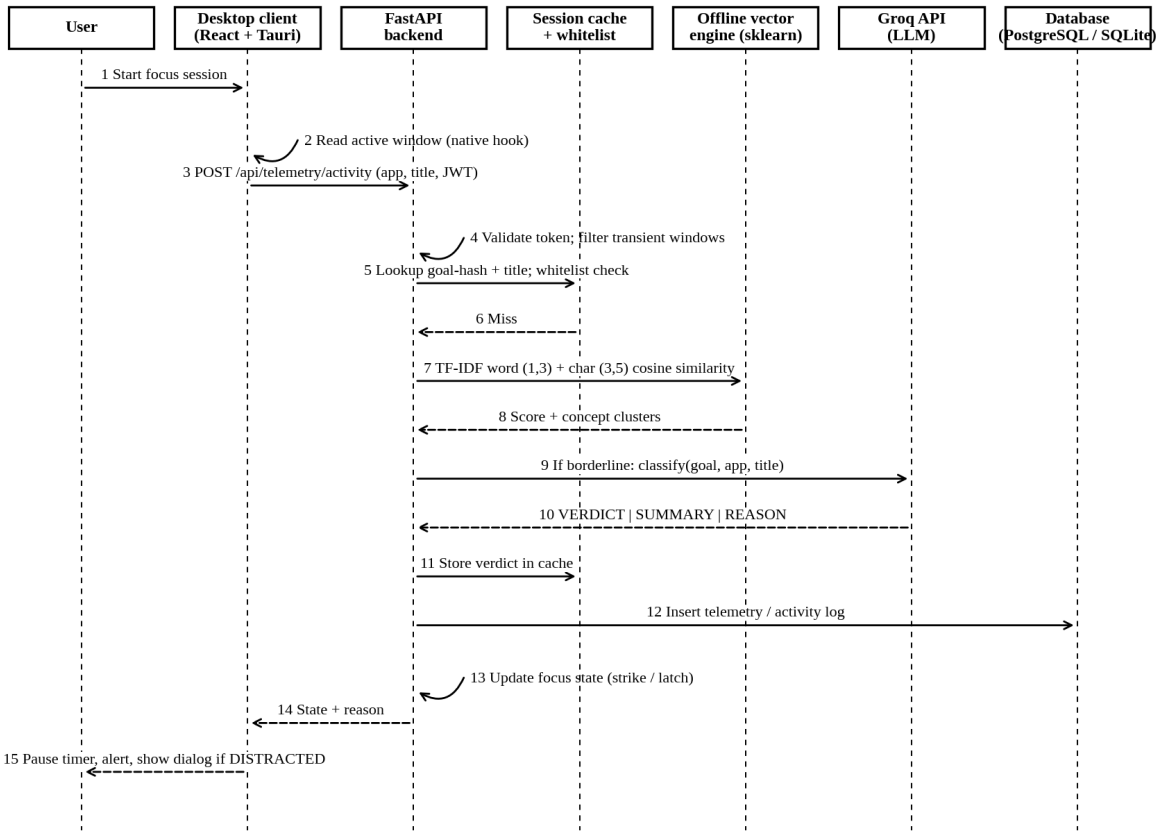


Fig 3.1 Authenticated window-classification sequence (hybrid offline + cloud engine)

3.1.2 Use Case Diagram

A Use Case Diagram depicts how users or external entities interact with a system, highlighting the functions the system provides. The primary actor is the User; the Friend (peer) takes part in the social use cases, and the external systems Supabase Auth and the Groq language model act as secondary actors. The use cases are: sign in; create, edit and delete a workspace (Mastery, Sprint or Quick Work); start, pause and resume a focus session; classify the active window; receive a distraction alert; allow a window for the session; view home metrics and streak; view the goal calendar and report; chat with the AI coach; change timer settings; enable camera tracking; reset all data; and, new in this version, **Manage Friends & Global Leaderboard**, **Send Direct Chat Messages** and **Hard Cascade Workspace Deletion**.

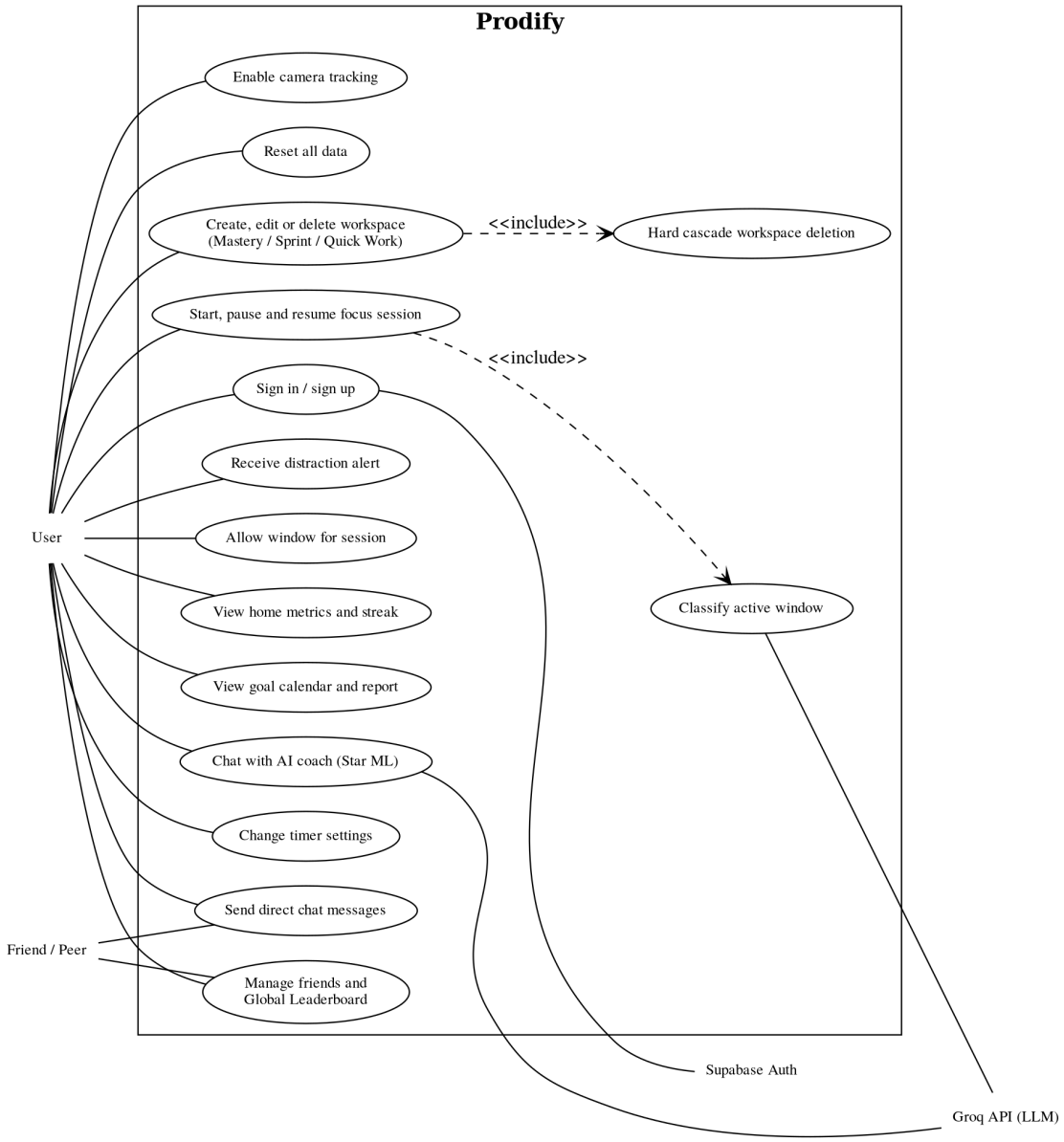
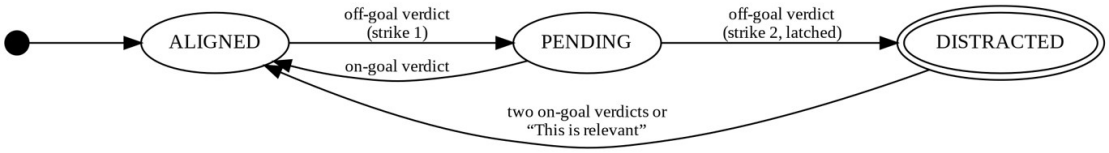


Fig 3.2 Prodify actors and use cases

3.1.3 State Diagram

A State Diagram shows the states an object or system can be in and the transitions between them. Prodify has two important state machines. The focus-state machine, kept by the back end, moves between **ALIGNED**, **PENDING** and **DISTRACTED**: the first off-goal window is a strike and moves the state to pending; a second consecutive off-goal window latches the state to distracted; and the state returns to aligned only after the classifier confirms that the user is back on goal-relevant windows or the user chooses “Allow for Session”. The timer state machine, kept by the client, moves between **IDLE**, **FOCUS_RUNNING**, **FOCUS_PAUSED**, **BREAK_RUNNING**, **BREAK_PAUSED** and **SESSION_COMPLETED**.

(a) Focus-state machine (back end)



(b) Timer state machine (client)

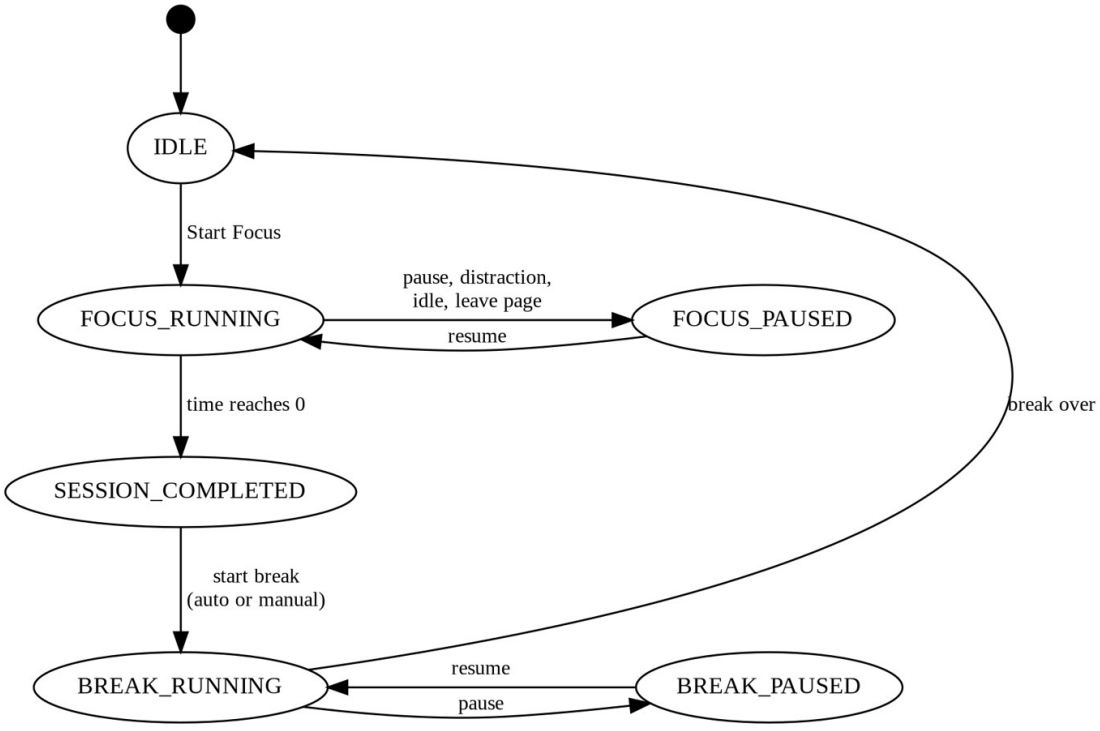


Fig 3.3 Focus and timer states

3.1.4 Activity Diagram

An Activity Diagram models the flow of activities in a process and is useful for showing decisions and loops. In Prodify the main activity flow is: sign in, choose or create a workspace, press Start Focus, work while the system observes the active window, handle any interruption (return to focus or allow the window), finish the focus session, save the session with its distraction data, take a break, and view the updated metrics. The flow repeats until the daily target is reached.

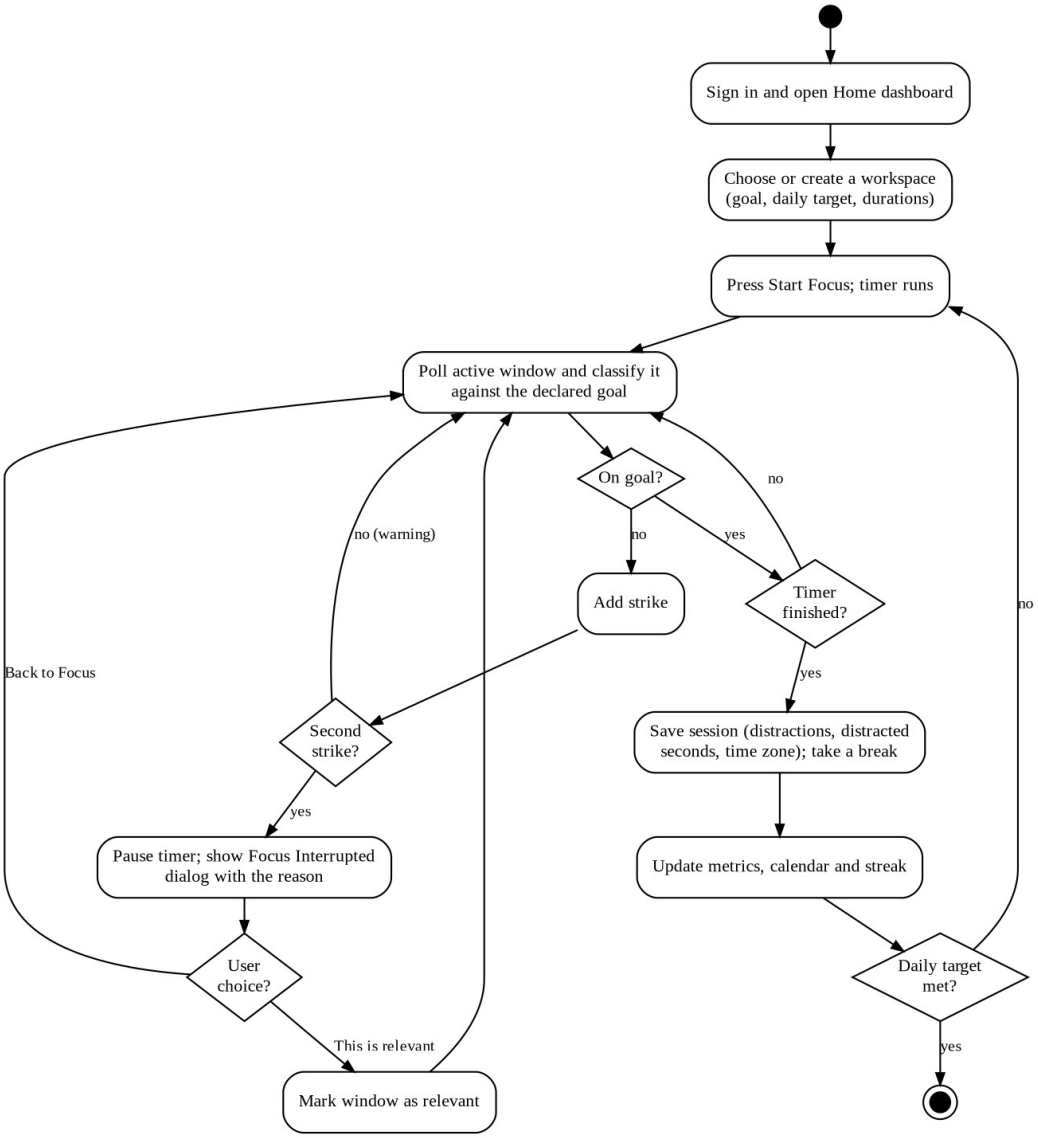
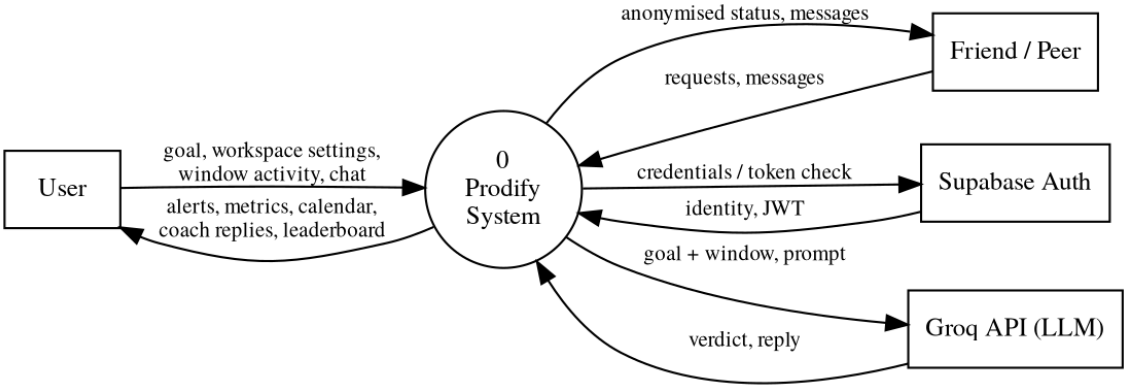


Fig 3.4 User focus-session activity

3.1.5 Data Flow Diagram (DFD)

A Data Flow Diagram shows how data moves through a system: the processes that transform it, the data stores that hold it and the external entities that send or receive it. At Level 0, Prodify is a single process between the User, the Friend and the external services (Supabase Auth and Groq). At Level 1 it is divided into eight processes – Authentication, Window Classification, Session Management, Metrics Computation, Star ML Coaching, **Friends and Global Leaderboard**, **Direct Chat** and **Hard Cascade Workspace Deletion** – which read and write the data stores D1 Users / Workspaces, D2 Sessions, D3 Activity logs and Telemetry, D4 Daily workspace logs and Metrics, D5 Chat history, **D6 Friendships & Direct Messages** and **D7 UserStats**. The cascade-deletion process purges D1 (the workspace), D2, D3, D4 and D5 for the deleted workspace and resets the streak held in D7 when no workspace remains.



Level 0 – Context diagram

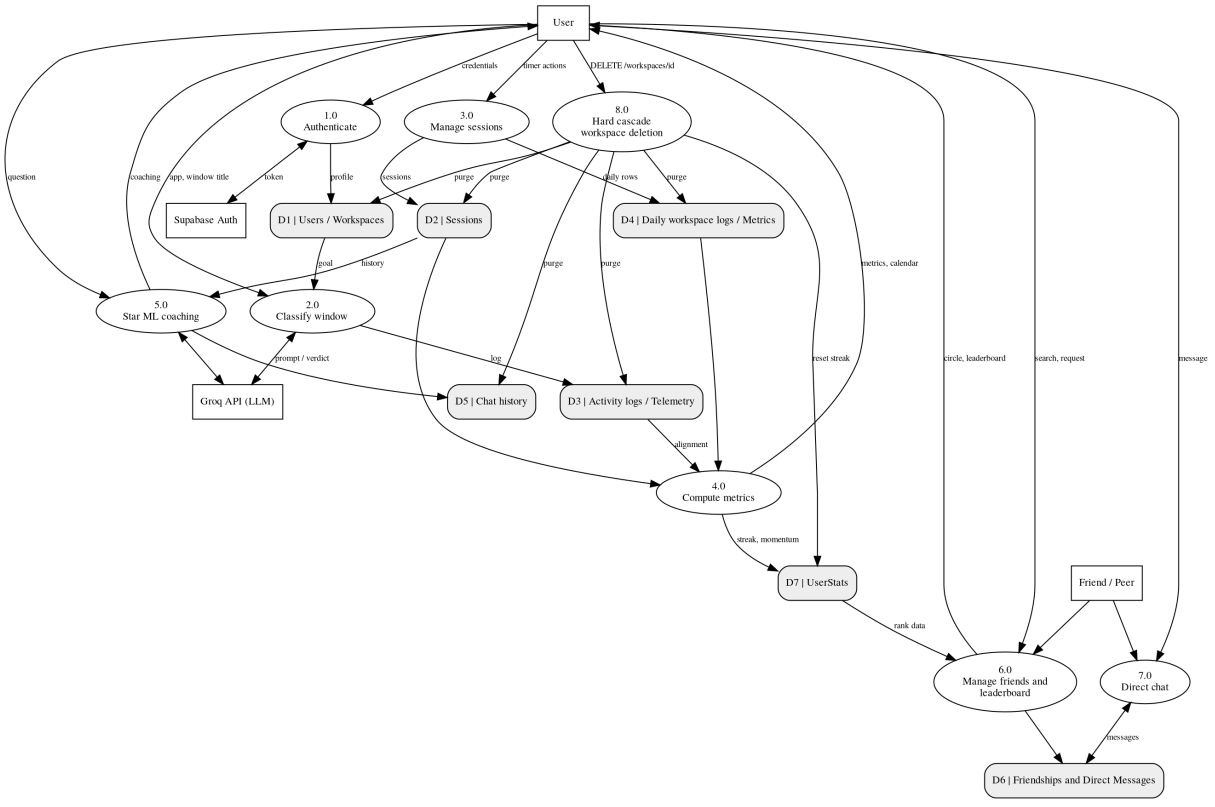


Fig 3.5 Level-one data flow diagram

3.1.6 Deployment Diagram

A Deployment Diagram illustrates the physical deployment of software components on hardware nodes. Prodify has three nodes. The user's Windows computer runs the Tauri application (a WebView hosting the React interface and the Rust core), together with the local SQLite fallback database when the cloud is unreachable. The Railway cloud hosts the FastAPI container, which is rebuilt each time the GitHub repository is updated. Supabase hosts the PostgreSQL database and the authentication service. The back end also communicates with the Groq API for language-model inference. All communication between nodes uses HTTPS (and secure WebSockets for camera frames).

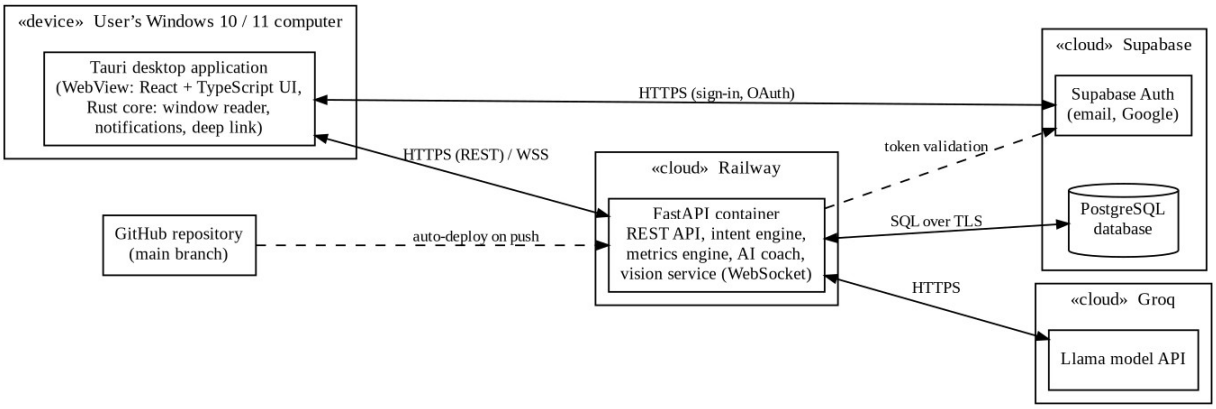


Fig 3.6 Deployment with Railway, Supabase and Groq

3.2 Metrics Engine Design

The metrics engine (`metrics_engine.py`) is a separate service module that computes every number shown in the interface from real logged data. It is designed around one building block, the day status, from which the calendar, the consistency score and the streak are derived. All calendar days are determined in the user's own time zone, which the client sends with each saved session.

Workspace Tracks and Daily Targets

Track (category)	Daily target rule	Effect on metrics
Mastery (`mastery`)	Fixed daily quota (for example 240 minutes per day)	Counted in the day status, consistency and the Active Streak
Sprint (`sprint`)	Dynamic: remaining target hours divided by the days remaining until the deadline	Counted in the day status, consistency and the Active Streak
Quick Work (`quick_work`)	`daily_target = 0` (enforced in `metrics_engine.py` and `crud.py`)	Logged minutes count towards volume and purity but never create unfulfilled-quota penalties and never break the streak

Day Status

For one workspace on one day, the status compares the minutes actually logged with the workspace's daily target:

Status	Condition	Meaning
RED	actual < 0.5 × target	Less than half of the target

Status	Condition	Meaning
YELLOW	0.5 × target ≤ actual < 1.0 × target	Partly done
GREEN	1.0 × target ≤ actual < 1.5 × target	Target met
GOLDEN	actual ≥ 1.5 × target	Target exceeded by half or more
No data	Day before the workspace existed, or a Quick Work track	Not counted (never shown as red)

On the Home dashboard the overall status of a day is the worst status among all workspaces that existed that day and carry a quota (RED < YELLOW < GREEN < GOLDEN), so one neglected workspace cannot be hidden by a successful one.

Timezone-Aware Aggregation

Session timestamps are stored in UTC. The function `get_local_date(utc_datetime, timezone)` converts a timestamp to the user's local calendar date (for example `Asia/Kolkata`), and every daily bucket uses that date. Because a local day does not coincide with a UTC day, queries fetch sessions with a ±1 day UTC buffer around the requested local range and then filter by local date, so that sessions near midnight are never lost or counted on the wrong day.

The Five Home Command Strip Metrics

The Focus Command Strip on the Home page (`HomePage.tsx`) shows five self-explanatory metrics, each with an educational popover (`MetricInfoPopover.tsx`). They replace the earlier names Today's Focus, Momentum, Deep Work Ratio, Peak Window and Global Streak.

Metric (UI name)	Formula / rule	Display
1. TODAY'S PROGRESS (formerly Today's Focus)	Σ Logged Focus Hours Today ÷ Σ Daily Target Quota Hours, over the active filter. COMPLETED and PARTIAL sessions (duration > 0) are counted and bucketed with `get_local_date` using the ±1 day UTC buffer.	`{logged}h / {target}h`
2. 7D CONSISTENCY (formerly Momentum)	Rolling 7-day composite of Quota Completion Volume (45%), Days Active Consistency (35%) and Focus Quality / Purity (20%), scaled 0–100.	`{score}%` with label: Strong 80–100, Solid 60–79, Shaky 40–59, Slipping <40; Ready when no data yet
3. FOCUS PURITY (formerly Deep Work Ratio)	((Total Session Seconds − Distracted Seconds − Penalty) ÷ Total Session Seconds) × 100	`{purity}%` and `{N} slips today`
4. PEAK HOUR (14D MODEL) (formerly Peak Window)	argmax over h in [0..23] of (Σ Aligned Seconds in hour h ÷ (Σ Total Logged Seconds in hour h + ε)) over a rolling 14-day telemetry window	`HH:00 – HH:00`, or `Calibrating <14d` until 14 days of history exist
5. ACTIVE STREAK (formerly Global Streak)	Consecutive calendar days that are GREEN or GOLDEN across all Mastery and Sprint quotas, counted backwards from today. Same-day grace: if today has no complete data yet, an in-progress day does not reset the counter to zero.	`{N} Days` and `Best: {M}d`

Consistency Score Components

Component	Weight	Definition
Quota Completion Volume	45%	(total minutes logged ÷ total target minutes) × 100, capped at 100, over the window
Days Active Consistency	35%	Average over the window of (workspaces touched that day ÷ active workspaces that day)
Focus Quality / Purity	20%	Focus Purity over the same window

Report Page (Report.tsx)

The refactored Report page presents analytics for the last 30 days (`GET /api/analytics/report?days=30`) in three parts:

- **Workspace Performance Leaderboard:** a table of all active tracks showing cumulative hours, pace percentage, focus purity, individual track streak and a one-click workspace launch button.
- **Distraction Intelligence:** full-width interactive **Platform Action Buttons**. Each button shows an untruncated platform name (for example YouTube, Antigravity or WhatsApp), the process of origin (for example Brave or App), a proportional time-loss bar, an event-count badge (`{N} events`) and the estimated time lost (`~{M}m lost`). Clicking a button expands an **inline accordion drawer** directly beneath it that reveals the exact window titles, slip counts and peak distraction hours (for example `Peak: 16:00`) for that platform; only one drawer needs to be open at a time.
- **Structured Goal Calendar (`StructuredGoalCalendar.tsx`):** a sharp 1px-bordered seven-column matrix (`gap-px bg-zinc-800`) with solid matte day cells. Completion dots are colour-coded: inactive, below 0.5× (red), 0.5–1× (yellow), 1–1.5× (emerald) and 1.5× or more (amber/gold), and today is marked by a 4px emerald dot for “in progress”. Clicking any date fills the **Day Inspector** panel with the exact minutes logged, the purity alignment percentage, the number of completed sessions and the contribution of each workspace.

3.3 Intent Engine Design

The intent engine (`intent_classifier.py`) decides whether the active window is on-goal. It combines cheap deterministic steps, an offline machine-learning stage and one language-model call so that the system is accurate, fast and resilient. No application name is hard-coded: an executable such as `chrome.exe`, `brave.exe`, `vlc.exe` or `code.exe` is never approved or rejected by its binary name alone (Executable Neutrality Principle).

1. **Transient OS window filter:** operating-system shell windows such as the task switcher, Task View, snap assist and empty titles are ignored, because they last for a moment and are not real destinations. The Prodify window itself is always considered aligned.
2. **Session verdict cache and “Allow for Session” override:** the cache key is a hash of the goal, application and window title; a hit returns the earlier verdict. If the user clicked “Allow for Session”, the window is added to a session-scoped whitelist and is accepted immediately.
3. **Offline multi-scale vector similarity (`evaluate_semantics_offline`):** Scikit-Learn `TfidfVectorizer` with sublinear term frequency builds word (1,3) n-gram and character (3,5) n-gram vectors for the declared goal and the window title, and cosine similarity measures their

closeness. The context is expanded with domain concept clusters (Software Engineering, Visual Arts, Audio Production, Data Science, Humanities, Writing). Recreational diversion signals such as gameplay walkthroughs, livestreams and comedy reels are considered only when they conflict with the stated objective. A clear score gives a verdict without any network call.

4. **Groq LLM contextual verdict (`evaluate_semantics`):** on a cache miss with a borderline offline score, the declared goal, allowed-task description, application name and window title are sent to the Groq-hosted model (qwen3 / Llama family) at temperature zero with a strict system prompt that recognises fluid multi-app workflows. It answers with `VERDICT: TRUE/FALSE | SUMMARY | REASON`.
5. **State machine:** a false verdict adds a strike (aligned → pending, then distracted); a true verdict moves the system back towards aligned, and a latched distraction is released only after the user returns to goal-relevant windows.
6. **User override:** "Allow for Session" whitelists the current window for the session and "Back to Focus" resumes the session.

If the cloud model is unreachable, the offline similarity verdict is used, so the engine keeps working; the model is run with a temperature of zero so that the same input produces the same verdict.

3.4 Star ML Predictive Engine Design

`star_ml.py` acts as an intelligent middle layer between the raw telemetry and the language-model coach. For each user it builds a feature table from the stored sessions – session duration, hour of day, number and timing of slips, and the share of focused seconds – and trains Scikit-Learn Random Forest models locally:

- **Attention Decay Wall:** a `RandomForestClassifier` learns at which elapsed minute of a session a slip becomes likely, and the point where predicted focus degrades sharply (for example after 47 minutes) is reported as the user's wall.
- **Chronobiological Peak Hour (14D model):** hourly aligned-time ratios from the rolling 14-day window are clustered to find the circadian peak window; the Home metric "Peak Hour" shows "Calibrating <14d" until enough data exists.
- **Burnout Risk Score:** a `RandomForestRegressor` estimates a score from 0 to 1 from time of day, length of continuous focus and recent load.

These mathematical findings are injected into the coaching prompt, so that the coach quotes the user's own wall, peak hour and burnout risk instead of generic advice.

3.5 Social Circle and Hard Cascade Deletion Design

The social module stores friendships (`PENDING`, `ACCEPTED`, `DECLINED`) and direct messages in two tables. Peer search (`GET /api/friends/search`) matches *@handle*, name or email. Presence is derived from recent activity (`last_active_at`) and an active session. Friends see only streak, momentum and a count of tracks per category, such as "2 Mastery · 1 Quick Work"; window titles and workspace names are never exposed. The global ranking orders users by `current_streak DESC, momentum_score DESC`.

Hard cascade deletion (`DELETE /workspaces/{id}`) removes the workspace and all dependent rows in one atomic transaction, recalculates the global streak and home metrics immediately, and invalidates the client cache. The database-level design is described in Appendix A (Section 9.2).

CHAPTER 4
IMPLEMENTATION & TESTING

4.1 Code

The source is divided into a back-end service and a front-end application. The implementation map below names the main modules. Configuration secrets (database URL, API keys) and generated dependency folders are intentionally omitted from the listing.

```
PRODIFY IMPLEMENTATION MAP
backend/
  main.py                 - application entry, routers, start-up
  app/api/                - auth, workspaces, telemetry, metrics,
                            analytics, ai_coach, users,
                            friends.py, leaderboard (REST routes)
  app/services/
    intent_classifier.py  - transient filter, cache + whitelist,
                            offline TF-IDF similarity, LLM verdict,
                            focus state machine
    metrics_engine.py     - day status, Today's Progress, 7D Consistency,
                            Focus Purity, Peak Hour (14D), Active Streak,
                            get_local_date, inner joins on active workspaces
    star_ml.py            - Random Forest Star ML: Attention Decay Wall,
                            Peak Hour, Burnout Risk; AI coach tools
    vision_engine.py      - optional MediaPipe presence detection
  app/models/             - models (ORM), crud (delete_workspace),
                            connection.py (PRAGMA foreign_keys = ON,
                            Supabase PostgreSQL / SQLite fallback)
  alembic/versions/       - database migrations
  verify_cascade_deletion.py - orphan verification script
frontend/
  src/pages/              - Home, Workspace, Report, FriendsPage.tsx,
                            Settings
  src/components/         - workspace card, StructuredGoalCalendar.tsx,
                            MetricInfoPopover.tsx, dialogs, coach panel,
                            timer settings
  src/store/cacheStore.ts - reactive TTL cache, instant invalidation
  src/lib/timerStore.ts   - persisted timer state and counters
  src/lib/globalSettingsStore.ts - global timer settings
  src/hooks/              - adaptive focus polling, idle detection
  src-tauri/              - Rust shell: active window, notifications,
                            deep link
```

Fig. 4.1 Prodify source structure and principal modules

The algorithms that carry most of the behaviour are summarised below as pseudo-code. They are simplified forms of the implementation and omit logging, error handling and database access.

Algorithm 1: Hybrid window evaluation

```
function evaluate(goal, window, session):
    if is_transient(window):            return NO_CHANGE
    key = hash(goal, window.app, window.title)
    if window in session.whitelist:     return ON_GOAL, "allowed for session"
    if key in cache:                    return cache[key]
    # executable neutrality: ignore window.app, judge the title
    score = tfidf_cosine(goal, window.title,
                         word_ngrams=(1,3), char_ngrams=(3,5),
                         clusters=domain_concept_clusters)
    if score >= HIGH:                                verdict = ON_GOAL
    elif score <= LOW and recreational_conflict:     verdict = OFF_GOAL
    else:                                            verdict = UNSURE   # borderline
    if verdict == UNSURE:
        verdict = groq_llm(goal, window)   # temperature 0
    cache[key] = verdict
    return verdict
```

Algorithm 2: Focus state update

```
function update_focus_state(workspace, window):
    verdict, reason = evaluate(goal, window)
    if verdict == ON_GOAL:
        if state == DISTRACTED:
            recovery += 1
            if recovery >= 2: state = ALIGNED; strikes = 0
        else:
            state = ALIGNED; strikes = 0
    else:
        recovery = 0; strikes += 1
        if strikes == 1: state = PENDING        # warning only
        if strikes >= 2: state = DISTRACTED     # latched, pause timer
    return state, reason
```

Algorithm 3: Day status and daily target

```
function daily_target(ws, today):
    if ws.category == "quick_work": return 0
    if ws.category == "sprint":
        return remaining_hours(ws) * 60 / max(1, days_until(ws.deadline))
    return ws.daily_target_minutes                  # mastery

function day_status(actual, target):
    if target is None or target == 0: return NO_DATA
    if actual < 0.5 * target: return RED
    if actual < target:       return YELLOW
    if actual < 1.5 * target: return GREEN
    return GOLDEN
```

Algorithm 4: Today's Progress and Focus Purity

```
function todays_progress(sessions, workspaces, tz):
    rows = sessions where status in (COMPLETED, PARTIAL) and duration > 0
           and local_date(start_time, tz) == today(tz)   # +-1 day UTC buffer query
    return sum(rows.focused_seconds)/3600, sum(targets(workspaces))/60

function focus_purity(total, distracted, penalty):
    return max(0, (total - distracted - penalty) / total) * 100
```

Algorithm 5: 7D Consistency, Peak Hour and Active Streak

```
function consistency(days):                     # last 7 local days
    volume = min(100, sum(actual)/sum(target)*100)
    active = mean(touched[d]/active_ws[d] for d in days)*100
    purity = focus_purity(window_totals)
    return 0.45*volume + 0.35*active + 0.20*purity

function peak_hour(telemetry_14d):
    if history_days < 14: return "Calibrating <14d"
    return argmax_h( aligned[h] / (total[h] + EPS) )

function active_streak(status_by_day, today):
    day = today
    if status[today] is NO_DATA or today in progress: day = today - 1
    streak = 0
    while status[day] in (GREEN, GOLDEN): streak += 1; day -= 1
    return streak
```

Algorithm 6: Star ML features and prediction

```
function star_ml(user):
    X = [duration, hour_of_day, slips, minute_of_first_slip, ...]
    clf = RandomForestClassifier().fit(X, slipped_in_next_segment)
    wall = first minute m where clf.predict_proba(m) crosses threshold
    reg = RandomForestRegressor().fit(X, burnout_target)
    risk = reg.predict(current_context)
    return {"decay_wall": wall, "peak_hour": peak_hour(...), "burnout": risk}
```

Algorithm 7: Hard cascade workspace deletion

```
function delete_workspace(db, user, ws_id):
    ws = get_owned_workspace(user, ws_id)        # 404 if not owner
    begin transaction
        delete WorkspaceMetrics, CoachingInteraction, WorkspaceChatHistory
               where workspace_id == ws_id         # explicit purge
        delete ws                                  # DB cascade removes
                                                   # Session, TelemetryLog,
                                                   # ActivityLog, DailyWorkspaceLog
        if count(user.workspaces) == 0: user_stats.current_global_streak = 0
    commit
    recompute_home_metrics(user); invalidate client cache
```

4.2 Test Case

Test cases describe a concrete trigger, preconditions, steps and expected behaviour. The cases below were carried out on Windows 11 with the back end running in WSL 2 and the database and authentication on Supabase (with the SQLite fallback used for the cascade script). Observed results record what was seen in the interface, in the back-end log and in the verification script output; the boundaries of each test are stated in the remark.

Test Case 1: Authentication and profile loading

- **Description:** confirm that a signed-in user is recognised by the back end and that the Home dashboard loads only that user's data.
- **Preconditions:** the user has an account; the back end is running and reachable.
- **Steps:** launch the desktop application, sign in, and observe the profile request (`GET /api/auth/me`) and the workspace list request.
- **Expected result:** the token is accepted, the profile is returned and the Home dashboard shows the user's own workspaces.
- **Observed result:** the back-end log shows `GET /api/auth/me` returning 200 OK and the Home dashboard loaded with the user's workspaces.
- **Remark:** PASS for the tested sign-in. Token refresh and expiry handling were not separately measured.

[Insert screenshot here: Sign-in screen on the dark Vercel Monolith canvas (#09090b) with matte white "Sign in" button, email field and Google sign-in option]

Fig. 4.2 Prodify sign-in screen (dark design system, email and Google sign-in)

Every request after sign-in carries the access token, and each database query is restricted to the user identified by that token, so one user's workspaces and sessions are never returned to another user.

Test Case 2: Workspace creation (three tracks)

- **Description:** create a workspace of each track type with a declared goal, timer durations, a daily commitment and a theme colour.

- **Preconditions:** the user is signed in.
- **Steps:** click New Workspace, choose Mastery, Sprint or Quick Work, enter the name, goal, durations, daily commitment (or target hours and deadline for Sprint), colour and the description of allowed apps and tasks, then click Create.
- **Expected result:** the workspace is saved and its card appears on the Home page; a Mastery track keeps its fixed quota, a Sprint track gets a computed daily quota, and a Quick Work track is stored with `daily_target = 0`.
- **Observed result:** the workspaces were created (201 Created) and their cards were displayed. During development, creation first failed with a 500 error because a database column added by a later migration (`theme_color`) did not yet exist; after applying the Alembic migration the test was repeated successfully.
- **Remark:** PASS after the migration. The case shows why migrations must be applied to the database before dependent code is used.

[Insert screenshot here: Create Workspace dialog with the three-track selector (Mastery / Sprint / Quick Work), goal text area, daily target, deadline and colour picker]

Fig. 4.3 Create Workspace dialog with Mastery, Sprint and Quick Work track selection

The daily commitment entered here becomes the target against which the day status, 7D Consistency and the calendar are calculated (for a Sprint track it is derived from the remaining hours and deadline; for Quick Work it is zero), so it is validated and stored together with the other timer settings.

Test Case 3: Focus timer and session counter

- **Description:** run a focus session and a break and show the progress of the day.
- **Preconditions:** a workspace exists with a daily commitment and a focus duration.
- **Steps:** open the workspace, press Start Focus, pause and resume, then let the focus session finish and observe the break timer.
- **Expected result:** the countdown runs, pause and resume work, the session dots and the “Session X of N today” label update and the break timer starts after a completed session.
- **Observed result:** the countdown ran, a break timer was started after the focus session, and the label and dots reflected the sessions completed that day (for example “Session 4 of 10 today”).
- **Remark:** PASS for the tested workspace. The number of sessions per day is the daily commitment divided by the focus duration.

[Insert screenshot here: Workspace timer with session dots, “Session X of N today”, live consistency, purity and time left today in JetBrains Mono]

Fig. 4.4 Workspace timer with session dots and live telemetry

The timer keeps its state in a persisted client store, so counters and the remaining time do not depend on a single page load, and break-over notifications are sent through the native layer.

Test Case 4: Distraction detection with the hybrid engine

- **Description:** interrupt the user when a window is unrelated to the declared goal, accept windows that the goal names, and let the user override a verdict for the session.
- **Preconditions:** a focus session is running with the goal “doing assignments using Gemini, Claude, Antigravity and VS Code”; the language-model service is reachable.
- **Steps:** open a social-media page (Instagram messages) in the browser and stay on it for two checks; then switch to Visual Studio Code and to Google Gemini in the browser; finally click “Allow for Session” on a flagged window.
- **Expected result:** the social-media window receives an off-goal verdict with a reason and moves the state to PENDING and then DISTRACTED; the editor and the named AI tool are accepted; an allowed window is accepted instantly.
- **Observed result:** the log shows a false verdict for the Instagram window (“social platform unrelated to the declared goal”), the state moving ALIGNED → PENDING (strike 1) → DISTRACTED (strike 2); Visual Studio Code and Google Gemini received true verdicts stating that the tools are listed in the goal.
- **Remark:** PASS for the tested windows. This is a small manual sample with one goal and is not an accuracy benchmark. The semantic test matrix of the Result chapter extends the same method to other goals.

[Insert screenshot here: Focus Interrupted dialog in rose accent showing the declared goal, the classifier reason and the buttons “Back to Focus” and “Allow for Session”]

Fig. 4.5 Focus Interrupted dialog with the reason and the Allow for Session override

The dialog shows the declared goal and the reason given by the classifier. The user may return to focus or allow the window, which whitelists it for the rest of the session.

Test Case 5: Metrics computation

- **Description:** return the Home metrics computed from stored sessions and check them against a hand calculation.
- **Preconditions:** a test Mastery workspace has a 120-minute daily target and 30 minutes of focus logged today with no distraction.
- **Steps:** call `GET /api/metrics/home` directly, then open the Home dashboard.
- **Expected result:** Today's Progress shows 0.5h / 2h, Focus Purity is 100%, today's calendar status is RED (30 of 120 minutes is below half the target), and 7D Consistency follows the weighted formula (45% × 25 + 35% × 100 + 20% × 100 ≈ 66, label Solid).
- **Observed result:** the response and the Focus Command Strip showed the same values as the hand calculation, with Peak Hour displaying "Calibrating <14d" for the new account.
- **Remark:** PASS for the example data, checked against a hand calculation of the formulas in Section 3.2.

[Insert screenshot here: Home Focus Command Strip with TODAY'S PROGRESS, 7D CONSISTENCY, FOCUS PURITY, PEAK HOUR (14D MODEL) and ACTIVE STREAK cards and an open metric info popover]

Fig. 4.6 Focus Command Strip and metrics response for the test workspace

A Focus Purity of 100% together with a RED day is consistent: the short time that was logged contained no distraction, but it was far below the target, so quality is high while volume is low.

Test Case 6: AI coach conversation with Star ML insights

- **Description:** answer a question about progress using the user's own workspace data and the personal model findings.
- **Preconditions:** the user is signed in and has a workspace with logged sessions.
- **Steps:** open the coach panel and ask how many hours per day are needed to reach the goal and when to schedule hard work.
- **Expected result:** the request succeeds and the reply uses the logged data (total focus time, sessions, consistency) and, when enough history exists, the user's Attention Decay Wall and Peak Hour, and asks for a target if none is set.
- **Observed result:** the chat endpoint returned 200 OK; the reply quoted the total focus logged and the number of sessions and asked the user to set a target and deadline before a daily plan could be given.
- **Remark:** PASS for the request and the use of stored data. The wording of replies varies because it is generated by a language model.

[Insert screenshot here: AI coach slide-over panel with a question about daily hours and a reply quoting logged hours, decay wall and peak hour]

Fig. 4.7 AI coach panel with Star ML findings

The coach is given the workspace settings, up to sixty days of session history and the Star ML findings through internal tools, so its answers refer to the user's real figures instead of generic advice.

Test Case 7: Hard cascade workspace deletion and zero-orphan verification

- **Description:** verify that deleting a workspace through `DELETE /workspaces/{id}` atomically purges all dependent rows, resets the global streak when no workspace remains, and leaves no ghost metrics.
- **Preconditions:** a test user with two workspaces (A and B), each with sessions, telemetry logs, activity logs, daily workspace logs, workspace metrics and coach chat history; `PRAGMA foreign_keys = ON` is active.
- **Steps:** run `verify_cascade_deletion.py`: record row counts for every child table; delete workspace A; recount; call `/api/metrics/home` and `/api/analytics/report`; delete workspace B; recount and check the streak.
- **Expected result:** after deleting A, no `Session`, `TelemetryLog`, `ActivityLog`, `DailyWorkspaceLog`, `WorkspaceMetrics` or `WorkspaceChatHistory` row of A remains and B is intact; after deleting B, `current_global_streak = 0`, and `/api/metrics/home` and `/api/analytics/report` contain no hours, quotas, calendar entries or distraction records of the deleted workspaces.
- **Observed result:** the script reported zero residual rows for every child table of the deleted workspaces, workspace B unaffected after A was deleted, the streak reset to 0 when no workspace remained, and zero ghost values in the home metrics and the report.
- **Remark:** PASS for the scripted multi-workspace scenario. Orphan counts were checked on the development database; the same cascade declarations apply to PostgreSQL.

[Insert screenshot here: terminal output of verify_cascade_deletion.py listing child-table row counts before and after deletion, all zero for the deleted workspace]

Fig. 4.8 Cascade deletion and orphan verification script output

The test relies on three mechanisms working together: `ondelete="CASCADE"` on the foreign keys, `PRAGMA foreign_keys = ON` in `connection.py` so that SQLite enforces them, and the explicit purge of tables without a cascade inside `crud.delete_workspace`.

Test Case 8: Social circle, handle search and real-time peer chat

- **Description:** verify friend requests, anonymised track badges, global leaderboard ordering and direct messaging.
- **Preconditions:** two accounts (User 1 and User 2) with unique *@username* handles; User 1 owns one Quick Work and two Mastery workspaces; the back end is running.
- **Steps:** User 1 searches for User 2 by handle, name and email (`GET /api/friends/search`) and sends a request; User 2 accepts it; open the circle table and the global leaderboard; User 1 opens the chat drawer and sends a message; User 2 polls for it.
- **Expected result:** the search returns the peer; after acceptance both appear in each other's circle with live status; User 1's row shows the badge "2 Mastery · 1 Quick Work" without any window title or workspace name; the global leaderboard is ordered by active streak and then momentum score with the top three on podium cards; the message appears in the other user's drawer within the three-second polling interval.
- **Observed result:** the request, acceptance, anonymised badge, ranking order and message delivery behaved as expected for the two test accounts.
- **Remark:** PASS for a two-user manual test. Many simultaneous users were not tested.



Fig. 4.9 Friends page: My Circle & Chat with the direct messaging drawer

4.3 Testing Approach

1. Manual functional testing

Each feature was exercised through the real interface during study sessions: sign-in, workspace creation, the timer, interruptions, the calendar, the report and the friends page. Defects found in this way, such as a calendar that showed squares on future dates and a timer that reset on refresh, were corrected and the feature was repeated.

2. API and log inspection

The REST endpoints were called directly and through the interface, and the back-end log was read to confirm status codes, classifier verdicts and state transitions. Metrics values were compared with hand-calculated results for test workspaces.

3. Database, migration and cascade checks

Alembic migrations were applied and the migration revision was checked against the head revision. Workspace deletion was exercised to find foreign-key conflicts, and the script-based orphan verification (Test Case 7) counts the rows of every dependent table before and after deletion. The data-reset function was checked by counting rows before and after.

4. Intent-engine inspection

A set of windows was opened for a declared goal: clearly on-goal tools, clearly off-goal pages and ambiguous windows such as a blank file-explorer window. Verdicts and reasons were read in the log. Particular attention was paid to false positives, because a wrongly interrupted user loses trust in the system, and one such case, caused by the operating-system task switcher, was removed with a transient-window rule.

5. Build and start-up checks

The front end was built with the TypeScript compiler and the production bundler, and the back end was started from a clean shell to confirm that the database tables are verified and the application starts.

6. Evidence separation

The results in this chapter come from manual and script-based checks on one development machine. They show that the tested paths work; they are not a measurement of accuracy, load capacity or learning benefit, and no such figures are claimed.

Verification summary

Table 4.1 Current verification and evidence status

Check	Status	Evidence boundary
Front-end production build	Passed	TypeScript compile and production bundle completed with no errors
Back-end start-up	Passed	Database tables verified, vision engine initialised, application startup complete
Database migrations	Applied	Alembic current revision equals head revision
Intent-engine verdicts	Observed	Off-goal window flagged, named tools accepted; small manual sample
Metrics endpoints	Checked	Values compared with hand calculation for test workspaces
Data reset	Checked	Row counts compared before and after in a development run
Automated test suite	Passed (Script-based multi-workspace cascade deletion & orphan verification suite executed)	`verify_cascade_deletion.py` executed on the development database; covers cascade deletion, streak reset and ghost-metric checks only
Social circle and chat	Checked	Two-account manual test of requests, anonymised badges, leaderboard order and messages
Load and performance benchmark	Not performed	No concurrent-user or response-time measurement
Cross-browser / accessibility audit	Not performed	Windows desktop client tested only
External plagiarism check	Not run	The checker report is to be attached by the student

Interpretation and remaining evaluation

The passed checks show that the main paths of Prodify work under the conditions tested. They do not measure classifier accuracy over many goals, behaviour under many simultaneous users, or whether the application improves a person’s results. These are the next steps: extending the automated test suite to the metrics and state-machine functions, a larger labelled set of windows for the intent engine, and a trial with several users over several weeks.

CHAPTER 5
RESULT

Screenshots of Prodify

The following are captures of the implemented application. They show the main stages of the user's journey, from signing in to reviewing the day's metrics, the social circle and the global leaderboard. All pages use the unified dark "Vercel Monolith" design system.



Fig. 5.1 Sign-in interface (email and Google sign-in on the obsidian canvas)

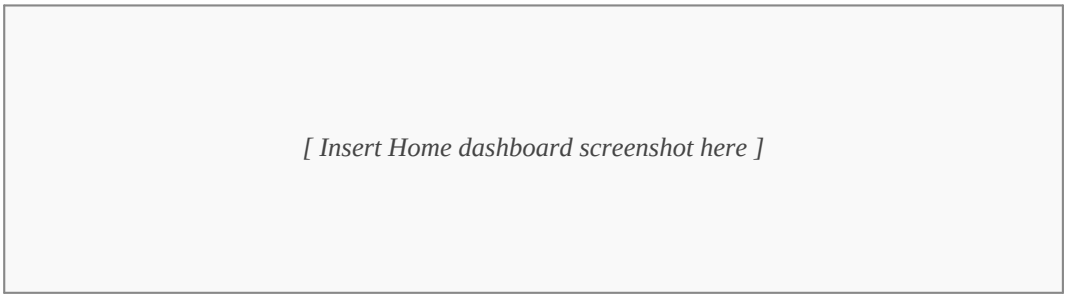


Fig. 5.2 Home dashboard with the Focus Command Strip: Today's Progress, 7D Consistency, Focus Purity, Peak Hour (14D Model) and Active Streak

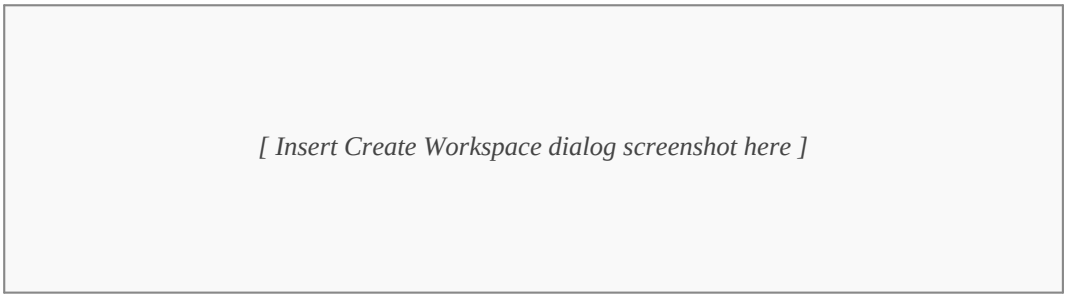


Fig. 5.3 Create Workspace dialog with the Mastery, Sprint and Quick Work tracks

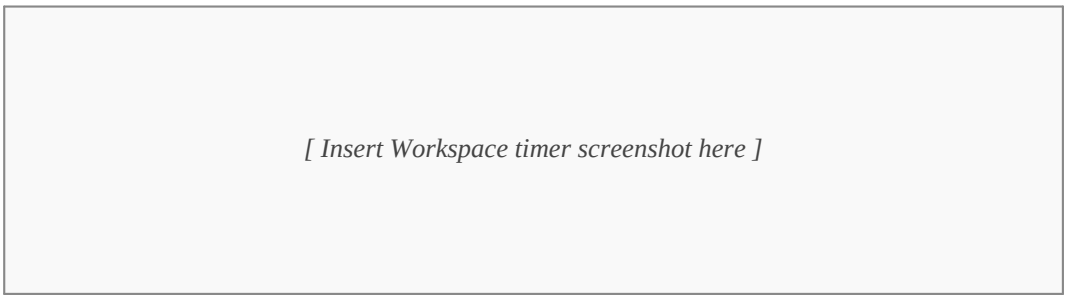


Fig. 5.4 Workspace timer with session dots, live purity, consistency and time left today

The sign-in page accepts the user's account, and the Home dashboard summarises today's progress, 7D Consistency, Focus Purity, Peak Hour and the Active Streak above the workspace cards. A workspace is created with its goal, track, timer durations, daily commitment, colour theme and a

plain-English description of allowed apps and tasks. The workspace screen shows the focus timer, the session dots and the live metrics.

Interruptions and Calendar

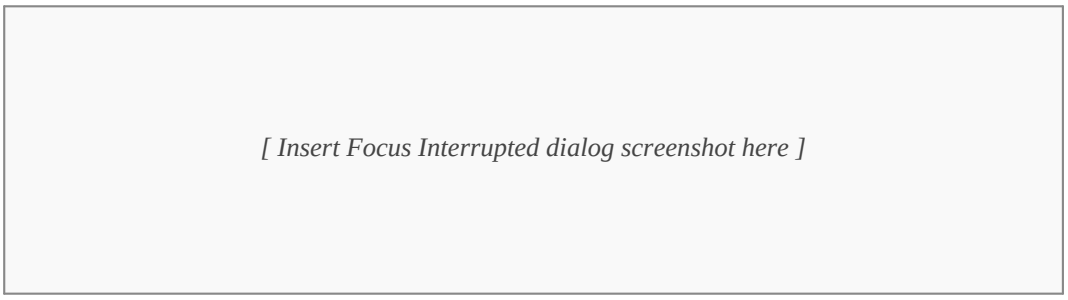


Fig. 5.5 Focus Interrupted dialog with the reason and the Back to Focus / Allow for Session buttons

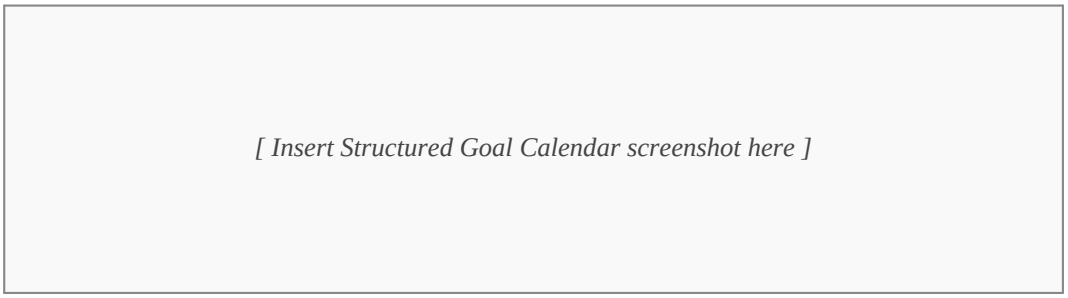


Fig. 5.6 Structured Goal Calendar: 1px seven-column matrix, 4px emerald dot for today, and the Day Inspector panel

When a distraction is confirmed the timer pauses and the dialog shows the declared goal and the reason. The goal calendar colours each day red, yellow, green or golden according to the minutes logged against the daily target; selecting a day opens the Day Inspector with the minutes logged, purity alignment, completed sessions and the contribution of each workspace.

AI Coach and Settings

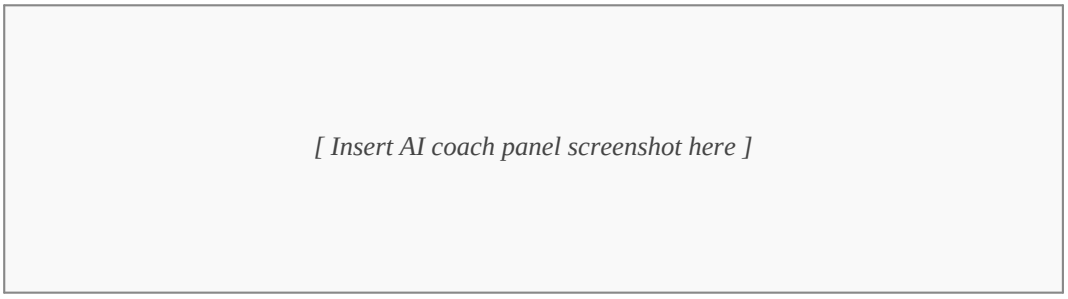


Fig. 5.7 AI coach panel with Star ML findings (Attention Decay Wall, Peak Hour, Burnout Risk)

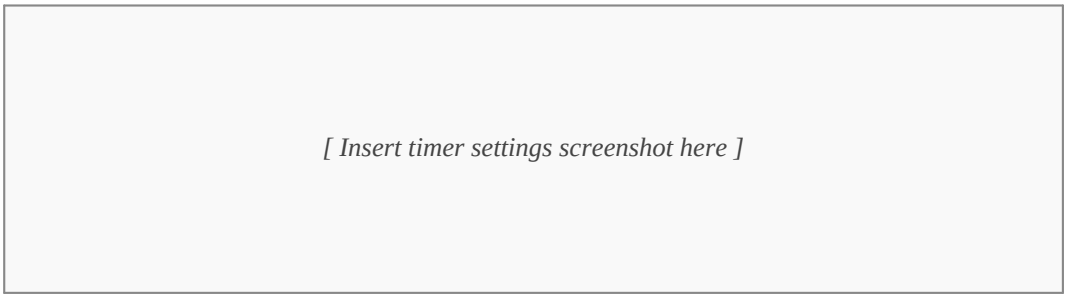


Fig. 5.8 Global timer settings

The coach panel answers questions about progress and planning using the user's own workspace data and personal model findings. The timer settings control automatic breaks and sessions, the long-break interval, alarm and focus sounds and the hour format for every workspace.

Reports



Fig. 5.9 Distraction Intelligence with platform action buttons and an expanded inline accordion drawer showing window titles and peak hour

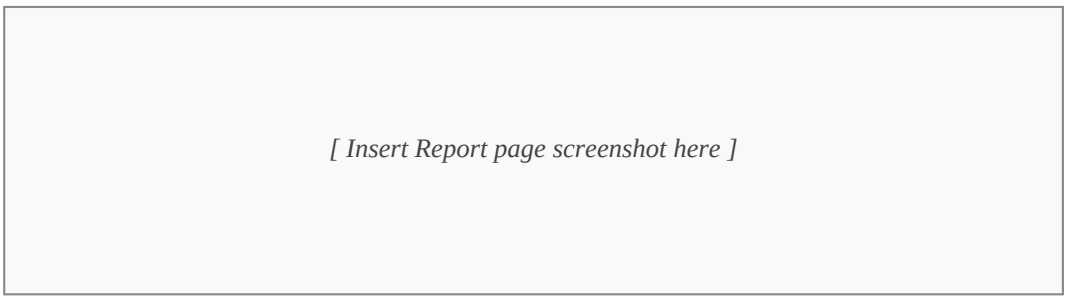


Fig. 5.10 Report page with the Workspace Performance Leaderboard and 30-day analytics

The Distraction Intelligence panel lists each platform with its process of origin, a proportional time-loss bar, an event count and the estimated minutes lost; clicking a platform expands the exact window titles, slip counts and peak distraction hours. The Report page shows the Workspace Performance Leaderboard with cumulative hours, pace, purity and streak for every track, together with the goal calendar.

Social Circle and Global Leaderboard

[Insert My Circle & Chat screenshot here]

Fig. 5.11 Friends page, My Circle & Chat: live status, handle editor, circle table with anonymised track summaries and the chat drawer

[Insert Global Leaderboard screenshot here]

Fig. 5.12 Global Leaderboard with Top-3 podium cards (gold, silver, bronze) and the searchable ranking table

The “My Circle & Chat” tab shows live presence, friend requests and anonymised track summaries such as “2 Mastery · 1 Quick Work”, and opens a slide-over drawer for real-time direct messages. The “Global Leaderboard” tab ranks users worldwide by Active Streak and 7D Consistency, and offers direct Add Friend and Chat buttons.

CHAPTER 6
CONCLUSION

6.1 Conclusion

The Prodify project set out to build a focus tool that understands what the user is trying to do, treats them fairly and measures progress honestly. By combining a native desktop shell, a web-based interface, a FastAPI back end, a PostgreSQL database with a SQLite fallback, an offline Scikit-Learn engine and a hosted large language model, the project delivers a working system in which a declared goal in plain English is enough to protect a work session. The application can tell the difference between a browser used to study and a browser used to scroll, and it explains every interruption with a short reason.

A central design decision was to separate judgement from control. A hybrid classifier – a transient-window filter, a session verdict cache with a user whitelist, an offline TF-IDF vector engine and a zero-temperature language model – supplies the judgement about a single window, while a deterministic strike-based state machine decides what the application does with that judgement. This made the behaviour predictable, fast and resilient when the cloud model is unavailable, and it allowed false alarms, such as those caused by the operating-system task switcher, to be removed by rules without weakening the semantic layer. Testing showed that goal-relevant tools named by the user are accepted and unrelated windows such as social media are flagged with an understandable reason.

The second important decision concerned metrics. Early versions showed several percentages that were hard to interpret or contradictory. They were replaced by a small, fully specified set with self-explanatory names: Today's Progress, 7D Consistency, Focus Purity, Peak Hour (14D Model) and Active Streak, supported by track-aware quotas for Mastery, Sprint and Quick Work workspaces. Because every number is computed from real logged data in the user's local time zone, the interface can show an honest zero or "Calibrating <14d" instead of a flattering guess.

The Star ML engine showed that small, personal machine-learning models are practical on a laptop: Random Forest models trained on the user's own sessions provide an Attention Decay Wall, a chronobiological peak hour and a Burnout Risk Score that make the coach's advice specific. The privacy-first social circle and global leaderboard add accountability without exposing window titles or workspace names.

The project also taught practical lessons in building a full-stack product: migrations must be reviewed and applied to every database before the code that depends on them is deployed; every payload field must be traced from the interface to the database to avoid silently dropped data; foreign-key enforcement must be switched on explicitly in SQLite for cascades to work; and any claim of a working feature must be verified by running it. Persistent timer state, local-day bucketing, secure token validation, per-user data isolation, hard cascade deletion with a script-based orphan check and a complete data-reset function were added in response to real problems found during use.

Security and privacy were treated as requirements. Users are authenticated with signed tokens and can access only their own data; only the application name and window title are recorded; camera detection is optional and off by default; friends see only anonymised summaries; and the user can erase everything. Deployment is automated through GitHub and Railway, and the Windows installer is produced with the Tauri build pipeline.

In conclusion, Prodify shows that classical machine learning and a large language model can be used responsibly inside a conventional software system to provide context-aware assistance. It is useful today as a focus companion, and its modular design leaves a solid base for the extensions described

below.

6.2 Future Scope

1. **Mobile Companion (iOS and Android):** Extending the Tauri codebase with Tauri Mobile to build a companion app that shows streaks, reports, the leaderboard and chat, and sends break reminders.
2. **Browser DOM Extension:** Adding page-level context such as the web address, page headings and video titles for even more precise classification of browser activity.
3. **OS-Level Kernel Focus Mode:** Integrating with operating-system focus and notification controls (and, where permitted, kernel-level hooks) so that notifications are silenced and distracting applications are suspended during a session.
4. **Wearable Biometric Integration:** Using heart-rate variability (HRV), sleep and activity data from wearables to refine the fatigue and burnout estimates of the Star ML engine.
5. **Smarter Coaching:** Weekly written summaries, goal planning that proposes sessions per day from the deadline and past pace, and proactive suggestions before the user burns out.
6. **Cross-Platform Desktop Support:** Extending the Tauri client to macOS and Linux.
7. **Calendar and Task Integration:** Reading exams and deadlines from calendars and task managers to set Sprint targets automatically.
8. **Team and Classroom Mode:** Dashboards for teachers and managers, with the consent of the users, to see class or team focus patterns.
9. **Gamification:** Badges, levels and rewards for consistent streaks, golden days and improvements in Focus Purity.
10. **Multi-Language Support:** Supporting goals and window titles in several languages, and a translated interface.
11. **Accessibility Improvements:** Screen-reader support, high-contrast themes, adjustable text size and keyboard-only navigation.

By incorporating these enhancements, Prodify can evolve from a focus timer with intelligent supervision into a personal attention-management platform that learns from each user and helps them build lasting habits.

CHAPTER 7
LIST OF CONTENTS

List Of Contents: -

Fig. 2.1 Prodify twelve-week development schedule 14

Fig 3.1 Authenticated window-classification sequence (hybrid offline + cloud engine) 16

Fig 3.2 Prodify actors and use cases 17

Fig 3.3 Focus and timer states 18

Fig 3.4 User focus-session activity 19

Fig 3.5 Level-one data flow diagram 20

Fig 3.6 Deployment with Railway, Supabase and Groq 21

Fig. 4.1 Prodify source structure and principal modules 26

Fig. 4.2 Prodify sign-in screen (dark design system, email and Google sign-in) 28

Fig. 4.3 Create Workspace dialog with Mastery, Sprint and Quick Work track selection 29

Fig. 4.4 Workspace timer with session dots and live telemetry 30

Fig. 4.5 Focus Interrupted dialog with the reason and the Allow for Session override 30

Fig. 4.6 Focus Command Strip and metrics response for the test workspace 31

Fig. 4.7 AI coach panel with Star ML findings 32

Fig. 4.8 Cascade deletion and orphan verification script output 32

Fig. 4.9 Friends page: My Circle & Chat with the direct messaging drawer 33

Fig. 5.1 Sign-in interface (email and Google sign-in on the obsidian canvas) 37

Fig. 5.2 Home dashboard with the Focus Command Strip: Today’s Progress, 7D Consistency, Focus Purity, Peak Hour (14D Model) and Active Streak 37

Fig. 5.3 Create Workspace dialog with the Mastery, Sprint and Quick Work tracks 37

Fig. 5.4 Workspace timer with session dots, live purity, consistency and time left today 37

Fig. 5.5 Focus Interrupted dialog with the reason and the Back to Focus / Allow for Session buttons . . . 38

Fig. 5.6 Structured Goal Calendar: 1px seven-column matrix, 4px emerald dot for today, and the Day Inspector panel 38

Fig. 5.7 AI coach panel with Star ML findings (Attention Decay Wall, Peak Hour, Burnout Risk) 38

Fig. 5.8 Global timer settings 39

Fig. 5.9 Distraction Intelligence with platform action buttons and an expanded inline accordion drawer showing window titles and peak hour 39

Fig. 5.10 Report page with the Workspace Performance Leaderboard and 30-day analytics 39

Fig. 5.11 Friends page, My Circle & Chat: live status, handle editor, circle table with anonymised track summaries and the chat drawer 40

Fig. 5.12 Global Leaderboard with Top-3 podium cards (gold, silver, bronze) and the searchable ranking table 40

Fig 9.1 Prodify entity-relationship diagram 52

CHAPTER 8
REFERENCES

8.1 References for Prodify

1. Books and Research Papers

- Newport, C. (2016). *Deep Work: Rules for Focused Success in a Distracted World.* Grand Central Publishing.
- Cirillo, F. (2018). *The Pomodoro Technique: The Life-Changing Time-Management System.* Currency.
- Mark, G., Gudith, D., & Klocke, U. (2008). The cost of interrupted work: More speed and stress. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '08).* ACM.
- Mark, G., Gonzalez, V. M., & Harris, J. (2005). No task left behind? Examining the nature of fragmented work. *Proceedings of CHI '05.* ACM.
- Maslach, C., & Jackson, S. E. (1981). The measurement of experienced burnout. *Journal of Organizational Behavior, 2*(2), 99–113.
- Brown, T. B., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems 33 (NeurIPS 2020).*
- Dubey, A., et al. (2024). The Llama 3 herd of models. arXiv:2407.21783.
- Lugaresi, C., et al. (2019). MediaPipe: A framework for building perception pipelines. arXiv:1906.08172.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.
- Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32.
- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management, 24*(5), 513–523.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval.* Cambridge University Press.

2. Software and API Documentation

- Tauri Contributors. Tauri v2 Documentation. https://v2.tauri.app
- Ramírez, S. FastAPI Documentation. https://fastapi.tiangolo.com
- SQLAlchemy Authors. SQLAlchemy Documentation. https://docs.sqlalchemy.org
- Alembic Authors. Alembic Documentation. https://alembic.sqlalchemy.org
- Scikit-learn Developers. Scikit-learn Documentation (TfidfVectorizer, RandomForestClassifier, RandomForestRegressor). https://scikit-learn.org
- SQLite Consortium. SQLite Documentation (Foreign Key Support). https://www.sqlite.org/docs.html
- Supabase Inc. Supabase Documentation (Database and Auth). https://supabase.com/docs
- Groq Inc. GroqCloud API Documentation. https://console.groq.com/docs
- Google. MediaPipe Solutions Guide. https://ai.google.dev/edge/mediapipe
- Meta Platforms. React Documentation. https://react.dev
- Poimandres. Zustand Documentation. https://zustand.docs.pmnd.rs

- Tailwind Labs. Tailwind CSS Documentation. https://tailwindcss.com/docs
- PostgreSQL Global Development Group. PostgreSQL Documentation. https://www.postgresql.org/docs
- Railway Corporation. Railway Documentation. https://docs.railway.com

3. Standards

- Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519. IETF.
- Hardt, D. (2012). The OAuth 2.0 Authorization Framework. RFC 6749. IETF.

CHAPTER 9
GLOSSARY AND APPENDICES

9.1 Glossary of Prodify

1. **Focus Session:** A timed period of work (for example 25 minutes) during which the user's activity is observed.
2. **Workspace:** A container for one goal, with its own daily target, timer settings, colour and history.
3. **Workspace Track:** One of the three workspace categories: Mastery, Sprint or Quick Work.
4. **Mastery:** A long-term, habit-driven track with a fixed daily quota that contributes to the Active Streak.
5. **Sprint:** A deadline-driven track whose daily quota is computed from the remaining target hours and the days until the deadline.
6. **Quick Work:** An ad-hoc track with `daily_target = 0`; its minutes count towards volume and purity but never create quota penalties or break a streak.
7. **Declared Goal:** The plain-English statement of what the user is working on, used to judge windows.
8. **Intent Engine:** The back-end service (`intent_classifier.py`) that decides whether the active window is relevant to the declared goal.
9. **Executable Neutrality Principle:** The rule that no application is approved or rejected by its binary name alone; windows are judged by their title and the goal.
10. **Semantic Classification:** Judging by meaning rather than by matching names or addresses against a list.
11. **TF-IDF:** Term Frequency–Inverse Document Frequency: a weighting that gives high value to terms that are frequent in a text but rare overall.
12. **N-gram:** A sequence of n consecutive words (word n-gram) or characters (character n-gram), used to capture phrases and spelling variants.
13. **Cosine Similarity:** A measure of the angle between two vectors; 1 means identical direction and 0 means unrelated.
14. **Domain Concept Cluster:** A set of related terms for a field (such as Software Engineering or Visual Arts) used to expand the goal context.
15. **Large Language Model (LLM):** A neural network trained on large amounts of text that can understand and generate language.
16. **Groq:** A cloud service that runs language models with very low latency.
17. **Allow for Session:** A user override that whitelists the current window for the rest of the session.
18. **State Machine:** A model with a fixed set of states and rules for moving between them.
19. **Strike:** A confirmed off-goal observation; two strikes in a row cause an interruption.
20. **Transient Window:** A short-lived system window, such as the task switcher, that is ignored by the classifier.
21. **Verdict Cache:** A store of earlier classification results that avoids repeated computation and model calls.
22. **Today's Progress:** Logged focus hours today divided by the daily target quota hours (formerly Today's Focus).

23. **7D Consistency:** A 0–100 rolling seven-day score of quota completion volume, days-active consistency and focus quality (formerly Momentum).
24. **Focus Purity:** The share of session time not lost to distraction or penalty (formerly Deep Work Ratio).
25. **Peak Hour (14D Model):** The hour of day with the highest aligned-time ratio over 14 days (formerly Peak Window); shown as Calibrating <14d until enough data exists.
26. **Active Streak:** The number of consecutive days on which all required daily quotas were met (formerly Global Streak).
27. **Star ML:** The predictive engine (`star_ml.py`) that trains Random Forest models on the user's sessions.
28. **Random Forest:** An ensemble of decision trees used for classification and regression.
29. **Attention Decay Wall:** The minute of a session after which the user's focus degrades sharply.
30. **Burnout Risk:** An estimate (0–1) of fatigue based on time of day, length of continuous focus and recent load.
31. **Chronobiological Peak:** The time of day at which the user's focus is naturally strongest.
32. **Handle (@username):** A unique public name used to search for and add friends.
33. **Anonymised Track Summary:** A count of a friend's tracks by category (for example "2 Mastery · 1 Quick Work") that hides workspace names and window titles.
34. **Hard Cascade Deletion:** Deleting a workspace together with all dependent rows in one atomic transaction so that no orphan data survives.
35. **Orphan Row / Ghost Metric:** A database row or displayed figure that still refers to a workspace that no longer exists.
36. **Pomodoro Technique:** A time-management method that alternates short focused sessions with breaks.
37. **Tauri:** A framework for building desktop applications with a Rust core and a web interface.
38. **FastAPI:** A Python web framework for building APIs.
39. **ORM (SQLAlchemy):** Object-relational mapper that lets code work with database rows as objects.
40. **Migration (Alembic):** A versioned script that changes the database structure.
41. **SQLite Fallback:** A local file database used automatically when the cloud database is unreachable.
42. **JWT:** JSON Web Token: a signed token that proves the identity of a user.
43. **OAuth:** A standard for signing in through a third party such as Google.
44. **Deep Link:** A special address that opens the desktop application, used to finish Google sign-in.
45. **WebSocket:** A persistent two-way connection, used here to stream camera frames.
46. **MediaPipe:** A Google library for face and body detection.
47. **Zustand:** A small state-management library for React.
48. **Cache Store:** The client-side reactive TTL cache (`cacheStore.ts`) with optimistic updates and instant invalidation.

49. **Vercel Monolith:** The unified dark design system of Prodify (obsidian canvas, 1px zinc borders, monospace telemetry, matte white buttons, emerald status accents).
50. **REST API:** A web interface in which resources are accessed with standard HTTP requests.

9.2 Appendix A: Data Model / ER Diagram

The entity-relationship diagram in Figure 9.1 shows the principal tables of the Prodify database. A user owns many workspaces and has one set of global statistics (`user_stats`); a workspace has many sessions, activity logs, daily workspace logs and coach chat messages; and a session has many telemetry logs and activity logs. Users are connected to each other through friendships and direct messages. Dashed lines mark tables that are purged explicitly by `crud.delete_workspace` rather than by a database cascade.

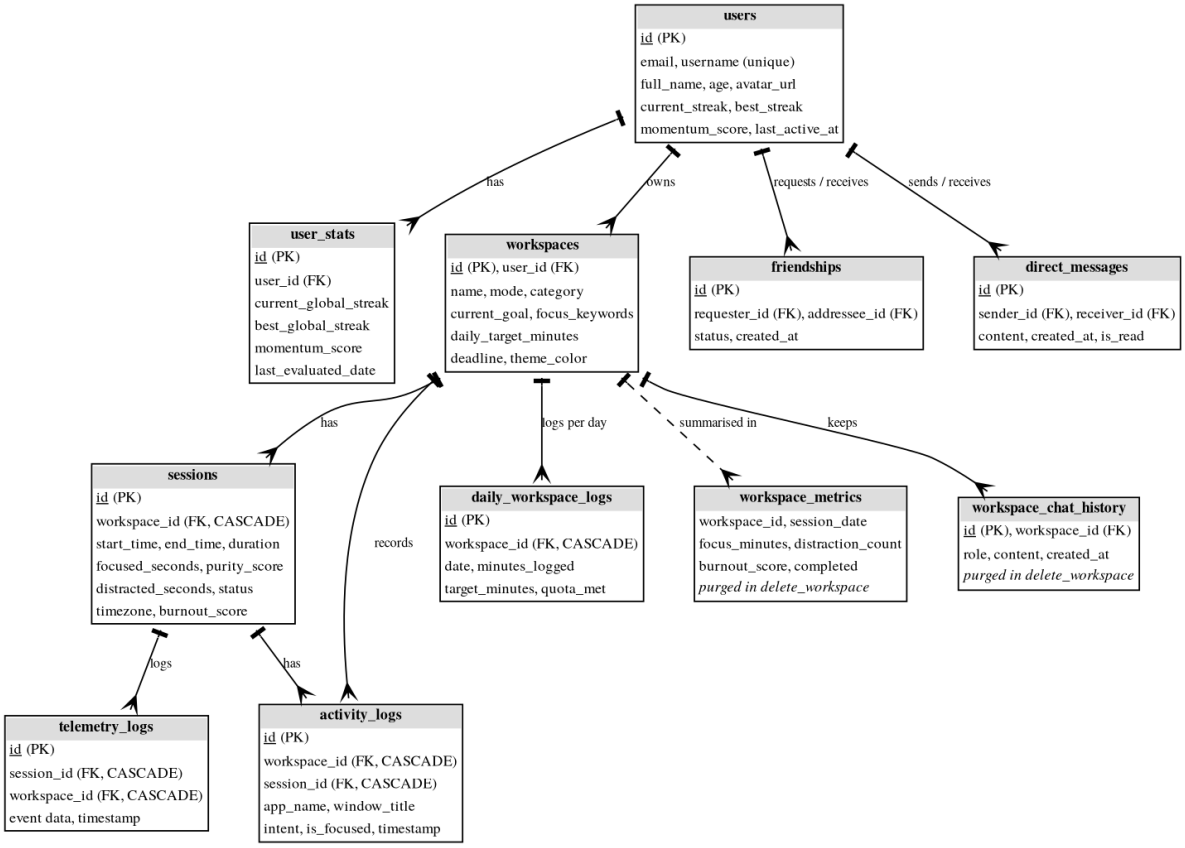


Fig 9.1 Prodify entity-relationship diagram

100% Hard Cascade Deletion & Zero-Orphan Architecture

Prodify guarantees that deleting a workspace leaves **no orphaned data and no ghost metrics**. The guarantee is built from four cooperating mechanisms:

1. **Foreign-key enforcement:** `PRAGMA foreign_keys = ON` is enforced for every connection in `connection.py`, so that SQLite, which ignores foreign keys by default, applies the same constraints as PostgreSQL.
2. **Declared cascades:** `ondelete="CASCADE"` is declared on `Session.workspace_id`, `TelemetryLog.workspace_id`, `TelemetryLog.session_id`, `ActivityLog.workspace_id`, `ActivityLog.session_id` and

`DailyWorkspaceLog.workspace_id`.

3. **Explicit transactional purge:** inside the same transaction, `crud.delete_workspace` deletes the `WorkspaceMetrics`, `CoachingInteraction` and `WorkspaceChatHistory` rows of the workspace, which are not covered by a database cascade.
4. **Inner joins against active workspaces:** `metrics_engine.py` joins sessions and logs to active `Workspace` rows (`Session.workspace_id.isnot(None)`), so that no figure can be computed for a workspace that no longer exists.

When a workspace is deleted, the global streak and home metrics are recomputed immediately (`current_global_streak` is reset to 0 when no workspace remains) and the client cache store is invalidated, so zero orphaned data or ghost metrics survive. This replaces the earlier design in which metrics could outlive a workspace.

Table: users

Column	Type	Description
id	Integer / UUID (PK)	Unique user identifier (from Supabase Auth)
email, username	String (unique)	Account e-mail and the unique *@handle*
full_name, age, avatar_url	String, Integer, String	Profile data
current_streak, best_streak	Integer	Streak counters shown to friends
momentum_score	Float	7D Consistency score used in the global ranking
last_active_at, created_at	DateTime	Presence and audit timestamps

Table: workspaces

Column	Type	Description
id	Integer (PK)	Unique workspace identifier
user_id	String (FK)	Owner of the workspace
name	String	Workspace name
mode, category	String	Track: mastery, sprint or quick_work
target_hours, deadline	Float, Date	Optional total goal and deadline (Sprint)
daily_target_minutes	Integer	Daily commitment in minutes (0 for quick_work); basis of the day status and calendar
work_duration, break_duration, long_break_duration, long_break_interval	Integer	Timer settings of the workspace
focus_keywords	Text	Description of allowed apps and tasks used by the intent engine
current_goal	String	Declared goal / intent statement
camera_enabled	Boolean	Whether camera presence detection is on (default off)
theme_color	String	Accent colour chosen for the workspace
current_streak, longest_streak	Integer	Stored track streak counters
created_at, updated_at	DateTime	Audit timestamps

Table: sessions

Column	Type	Description
id	Integer (PK)	Session identifier
workspace_id	Integer (FK, ON DELETE CASCADE)	Owning workspace
session_type, status	String	FOCUS or break; ACTIVE, COMPLETED or PARTIAL
start_time	DateTime	UTC start of the session
end_time	DateTime	UTC end of the session
duration	Integer	Session length in seconds
focused_seconds	Integer	Seconds aligned with the goal
distracted_seconds	Float	Seconds lost to distraction (used by Focus Purity)
purity_score	Float	Focus Purity of the session (0–100)
distraction_count	Integer	Number of distractions (slips) in the session
pause_reason	String	Reason recorded when a session is paused
burnout_score	Float	Burnout risk estimate (0–1) at completion
timezone	String	IANA time zone of the user, for local-day bucketing

Table: activity_logs

Column	Type	Description
id	Integer (PK)	Log identifier
workspace_id	Integer (FK, ON DELETE CASCADE)	Owning workspace
session_id	Integer (FK, ON DELETE CASCADE)	Owning session
app_name, window_title	String	Active application and its window title
intent	String	Short summary of what the window was judged to be
is_focused	Integer (0/1)	Verdict: 1 on-goal, 0 off-goal
timestamp	DateTime	Time of the observation

Table: telemetry_logs

Column	Type	Description
id	Integer (PK)	Log identifier
session_id	Integer (FK, ON DELETE CASCADE)	Owning session
workspace_id	Integer (FK, ON DELETE CASCADE)	Owning workspace
event data	Text / JSON	Per-event telemetry (similarity score, verdict reason)
timestamp	DateTime	Time of the event

Table: daily_workspace_logs

Column	Type	Description
id	Integer (PK)	Row identifier
workspace_id	Integer (FK, ON DELETE CASCADE)	Owning workspace
date	Date	Local-timezone day bucket
minutes_logged	Integer	Minutes logged that day
target_minutes	Integer	Daily target for that day
quota_met	Boolean	Whether the quota was reached

Table: user_stats

Column	Type	Description
user_id	String (FK)	Owner of the statistics row
current_global_streak	Integer	Active Streak across all Mastery and Sprint quotas
best_global_streak	Integer	Longest streak reached
momentum_score	Float	Latest 7D Consistency score
last_evaluated_date	Date	Local date on which the streak was last evaluated

Table: workspace_metrics

Column	Type	Description
workspace_id	Integer	Workspace identifier (purged explicitly by `crud.delete_workspace`)
workspace_name	String	Name kept with the row for reporting
session_date	Date	Day of the session
focus_minutes	Integer	Minutes of focus logged
distraction_count	Integer	Distractions in that session
burnout_score	Float	Burnout estimate
time_of_day, mode, completed	String / Boolean	Hour bucket, session mode and completion flag

Table: friendships

Column	Type	Description
id	Integer (PK)	Row identifier
requester_id	String (FK to users)	User who sent the request
addressee_id	String (FK to users)	User who received the request
status	String	PENDING, ACCEPTED or DECLINED
created_at	DateTime	Time of the request

Table: direct_messages

Column	Type	Description
id	Integer (PK)	Message identifier
sender_id	String (FK to users)	Sender
receiver_id	String (FK to users)	Receiver
content	Text	Message text
created_at	DateTime	Time sent
is_read	Boolean	Whether the receiver has read the message

Further tables hold the AI coach conversation per workspace (`workspace_chat_history`) and coaching interactions (`coaching_interactions`); both are purged with the workspace. Every schema change is expressed as an Alembic migration, so that the same structure can be created on a development and a production database.

9.3 Appendix B: Operation and Protocol

This appendix describes how the desktop client and the back end communicate and which services take part.

B.1 Main API Endpoints

Endpoint	Purpose
/api/auth/me	Returns the profile of the signed-in user
/workspaces	List and create workspaces (Mastery, Sprint, Quick Work)
/workspaces/{id}	Update one workspace; `DELETE` performs the hard cascade deletion
/workspaces/{id}/metrics	Store the metrics row of a finished session
/workspaces/{id}/chat-history	Load the AI coach conversation of a workspace
/api/telemetry/activity	Classify the active window with the hybrid engine and update the focus state
/api/telemetry/sessions	Save a completed session; list recent sessions
/api/metrics/home	Today’s Progress, 7D Consistency, Focus Purity, Peak Hour (14D), Active Streak and calendar for the Home page
/api/metrics/workspace/{id}	Consistency, purity, streaks and calendar for one workspace
/api/analytics/report?days=30	30-day goal calendar, Workspace Performance Leaderboard and Distraction Intelligence
/api/analytics/performance-matrix	Multi-dimensional efficiency and circadian heatmaps
/api/analytics/distracting-apps	Ranking of applications that distract most
/api/analytics/distraction-heatmap	Distractions by weekday and hour
/api/ai-coach/chat	Send a message to the AI coach and receive a reply
/api/ml/burnout-prediction	Burnout-risk estimate from the hour of day and focus minutes
/api/friends/leaderboard	Connected friends circle with live status and anonymised track summaries
/api/leaderboard/global	Worldwide ranking by Active Streak and 7D Consistency
/api/friends/search	Search peers by @handle, name or e-mail
/api/friends/requests	List, send, accept and decline friend requests (GET / POST)
/api/friends/messages/{friend_id}	Poll and send direct messages with a friend (GET / POST)
/api/users/me/reset-data	Erase all of the user’s data

B.2 Sign-in Protocol

With email sign-in, the credentials are sent to Supabase Auth, which returns a signed JSON Web Token. With Google sign-in, the application opens the system browser; after the user approves, Supabase redirects to a deep link that reopens the desktop application with the session. In both cases the token is sent in the Authorization header of every API request. The back end verifies the signature and expiry of the token and uses the user identifier inside it for every database query, so that one user can never read another user’s workspaces or sessions.

B.3 Activity and Session Protocol

While a focus session runs, the client sends a message only when the active window changes. The response carries the focus state (aligned, pending or distracted) and the reason. When the timer finishes, the client saves the session with its length, focused and distracted seconds, distraction count, burnout estimate and the user's time zone; the metrics are then recalculated on request from these stored values.

B.4 Use Case Scenarios

1. **Scenario 1 – Student working on assignments:** a student declares that they will use Gemini, Claude and VS Code to build a project. Prodify accepts those tools, but interrupts when Instagram is opened, showing the reason.
2. **Scenario 2 – Deadline sprint:** a developer creates a Sprint workspace with a target of 40 hours before a deadline. The daily quota is computed from the days remaining, and 7D Consistency and the calendar show whether the week is on track.
3. **Scenario 3 – Long-term learning:** a learner builds a Mastery workspace for daily practice. The streak, golden days and the Peak Hour show how the habit is forming and when to schedule the hardest work.
4. **Scenario 4 – Quick assignment:** a user logs a one-off Quick Work task; the minutes count towards volume and purity but never break the streak.
5. **Scenario 5 – Art study:** an art student watches a figure-drawing tutorial on YouTube; the title aligns with the declared goal, so no interruption occurs.
6. **Scenario 6 – Friends:** two friends add each other by handle, see each other's streak and anonymised track badges, and chat from the drawer.

B.5 Intent Classification Process

Prodify uses a hybrid classifier. After the transient-window filter and the cache and whitelist check, an offline Scikit-Learn TF-IDF engine (word (1,3) and character (3,5) n-grams, cosine similarity and domain concept clusters) scores the window title against the declared goal. For borderline scores the engine sends a structured prompt containing the declared goal, the description of allowed tasks, the application name and the window title to a hosted language model (qwen3 / Llama family on Groq) run at zero temperature, and asks for a verdict, a one-line summary and a reason. The prompt instructs the model to judge by the user's own words, to treat tools named in the goal as allowed, and to be tolerant of informal, run-on goal text. Responses are parsed, cached and logged for the user's distraction report. Prodify does not send screenshots.

B.6 Security and Privacy Measures

- Supabase Auth with email and Google sign-in; signed JSON Web Tokens validated on every request.
- Every database query is restricted to the authenticated user's identifier.
- Secrets such as database URLs and API keys are kept in environment variables and not in the source code.
- Only application names and window titles are recorded; camera tracking is optional and off by default.

- Social views are anonymised: only streak, momentum and track-category counts are shared with friends.
- A consent screen on first use, hard cascade workspace deletion, and a Reset All Data function that erases the user's workspaces, sessions and logs.

B.7 Third-Party Services and Libraries

Service / Library	Use in Prodify
Supabase	PostgreSQL database hosting and authentication (email, Google)
SQLite	Local fallback database for offline resilience
Groq API	Low-latency inference of language models for contextual window verdicts and AI coaching
Scikit-Learn / NumPy	Offline TF-IDF vector similarity and Random Forest models (Star ML)
Google MediaPipe	Face-presence detection for the optional camera feature
Railway	Hosting and continuous deployment of the FastAPI back end
GitHub	Source control; pushes to the main branch trigger deployment